

```

1  // File: backend/chasing_points.api
2  syntax = "v1"
3
4  // ===== 通用类型 =====
5  type UserInfo {
6      Id    int64 `json:"id"`
7      Phone string `json:"phone"`
8      Nickname string `json:"nickname"`
9      Avatar string `json:"avatar"`
10     Status int `json:"status"`
11     CreatedAt string `json:"created_at"`
12 }
13
14 // ===== 认证相关 =====
15 // 发送短信验证码
16 type SendSmsReq {
17     Phone string `json:"phone"`
18     Scene string `json:"scene,optional,default=login" // login/bind
19 }
20
21 type SendSmsResp {
22     Success bool `json:"success"`
23     Message string `json:"message"`
24 }
25
26 // 手机号登录
27 type LoginReq {
28     Phone string `json:"phone"`
29     SmsCode string `json:"sms_code"`
30 }
31
32 type LoginResp {
33     Success bool `json:"success"`
34     AccessToken string `json:"access_token"`
35     RefreshToken string `json:"refresh_token"`
36     ExpiresIn int64 `json:"expires_in"`
37     UserInfo *UserInfo `json:"user_info"`
38 }
39
40 // OAuth登录
41 type LoginByOAuthReq {
42     Provider string `json:"provider" // huawei
43     OpenId string `json:"open_id"`
44     UnionId string `json:"union_id,optional"`
45     NickName string `json:"nick_name,optional"`
46     AvatarUrl string `json:"avatar_url,optional"`
47     Platform string `json:"platform,optional"`
48 }
49
50 type LoginByOAuthResp {

```

```

51     Success    bool    `json:"success"
52     AccessToken string `json:"access_token"
53     RefreshToken string `json:"refresh_token"
54     ExpiresIn  int64   `json:"expires_in"
55     NeedBindPhone bool   `json:"need_bind_phone"
56     UserInfo   *UserInfo `json:"user_info"
57 }
58
59 // 绑定手机号
60 type BindPhoneReq {
61     Phone string `json:"phone"
62     SmsCode string `json:"sms_code"
63 }
64
65 type BindPhoneResp {
66     Success    bool    `json:"success"
67     Message    string  `json:"message"
68     MergedAccount bool `json:"merged_account"
69 }
70
71 // ===== 服务定义 =====
72 @server (
73     prefix: /api/auth
74     group: auth
75 )
76 service chasing_points-api {
77     @doc "发送短信验证码"
78     @handler SendSms
79     post /send-sms (SendSmsReq) returns (SendSmsResp)
80
81     @doc "手机号登录/注册"
82     @handler Login
83     post /login (LoginReq) returns (LoginResp)
84
85     @doc "OAuth登录"
86     @handler LoginByOAuth
87     post /login-by-oauth (LoginByOAuthReq) returns (LoginByOAuthResp)
88 }
89
90 // ===== 公开接口 =====
91 // 正在进行的对局列表项
92 type OngoingMatchItem {
93     Id      int64 `json:"id"
94     GameType int   `json:"game_type"
95     GameTypeName string `json:"game_type_name"
96     Player1Id int64 `json:"player1_id" // 玩家1(创建者)ID
97     Player1Name string `json:"player1_name"
98     Player1Avatar string `json:"player1_avatar"
99     Player2Id int64 `json:"player2_id" // 玩家2(对手)ID, 0表示非注册用户
100    Player2Name string `json:"player2_name"
    
```

```

101   Player2Avatar string `json:"player2_avatar"
102   Player1Score  int  `json:"player1_score"
103   Player2Score  int  `json:"player2_score"
104   CurrentRound  int  `json:"current_round"
105   MatchTime     string `json:"match_time" // 开始时间 ISO格式
106   DurationSeconds int64 `json:"duration_seconds" // 对局已持续秒数，由服务端计算
107 }
108
109 // 获取正在进行的对局列表请求
110 type GetOngoingMatchesReq {
111     Page  int `form:"page,default=1"
112     PageSize int `form:"page_size,default=20"
113 }
114
115 // 获取正在进行的对局列表响应
116 type GetOngoingMatchesResp {
117     Success bool      `json:"success"
118     Total  int64      `json:"total"
119     List  []OngoingMatchItem `json:"list"
120 }
121
122 // 公开对局详情请求（观战模式）
123 type GetPublicMatchDetailReq {
124     MatchId int64 `form:"match_id"
125 }
126
127 // 局记录
128 type RoundRecord {
129     RoundNumber int `json:"round_number"
130     Player1Score int `json:"player1_score"
131     Player2Score int `json:"player2_score"
132     Winner      int `json:"winner" // 1=玩家1胜 2=玩家2胜
133 }
134
135 // 公开对局详情数据（第三方视角）
136 type PublicMatchDetailData {
137     Id          int64 `json:"id"
138     GameType    int   `json:"game_type"
139     Status      int   `json:"status" // 1=进行中 2=已完成 3=已取消
140     ServerRevision int64 `json:"server_revision"
141     Player1Id   int64 `json:"player1_id"
142     Player1Name string `json:"player1_name"
143     Player1Avatar string `json:"player1_avatar"
144     Player2Id   int64 `json:"player2_id"
145     Player2Name string `json:"player2_name"
146     Player2Avatar string `json:"player2_avatar"
147     Player1Score int   `json:"player1_score"
148     Player2Score int   `json:"player2_score"
149     CurrentFramePlayer1Score int `json:"current_frame_player1_score,optional"
150     CurrentFramePlayer2Score int `json:"current_frame_player2_score,optional"

```

```

151   CurrentFrameStarted  bool    `json:"current_frame_started,optional"
152   CurrentRound         int     `json:"current_round"
153   TotalRounds          int     `json:"total_rounds"
154   DurationSeconds      int64   `json:"duration_seconds"
155   Rounds                []RoundRecord `json:"rounds" // 局记录
156   CreatedAt            string  `json:"created_at"
157 }
158
159 // 公开对局详情响应
160 type GetPublicMatchDetailResp {
161     Success bool    `json:"success"
162     Match   *PublicMatchDetailData `json:"match"
163 }
164
165 @server (
166     prefix: /api/public
167     group: public
168 )
169 service chasing_points-api {
170     @doc (
171         summary: "获取正在进行的对局列表"
172         description: "获取平台所有正在进行的对局列表，公开接口无需登录"
173     )
174     @handler GetOngoingMatches
175     get /matches/ongoing (GetOngoingMatchesReq) returns (GetOngoingMatchesResp)
176
177     @doc (
178         summary: "获取段位排行榜"
179         description: "获取按排位分排序的用户排行榜，包含前三名和分页列表，公开接口无需登录"
180     )
181     @handler GetLeaderboard
182     get /rank/leaderboard (GetLeaderboardReq) returns (GetLeaderboardResp)
183
184     @doc (
185         summary: "获取公开对局详情"
186         description: "获取对局详情（观战模式），公开接口无需登录，使用第三方视角"
187     )
188     @handler GetPublicMatchDetail
189     get /match/detail (GetPublicMatchDetailReq) returns (GetPublicMatchDetailResp)
190 }
191
192 @server (
193     prefix: /api/auth
194     group: auth
195     jwt: Auth
196 )
197 service chasing_points-api {
198     @doc "绑定手机号"
199     @handler BindPhone
200     post /bind-phone (BindPhoneReq) returns (BindPhoneResp)

```

```

201  }
202
203  // ===== 用户相关 =====
204  // 更新昵称
205  type UpdateNicknameReq {
206      Nickname string `json:"nickname"`
207  }
208
209  type UpdateNicknameResp {
210      Success bool `json:"success"`
211      Message string `json:"message"`
212      UserInfo *UserInfo `json:"user_info"`
213  }
214
215  // 获取用户信息
216  type GetUserInfoResp {
217      Success bool `json:"success"`
218      UserInfo *UserInfo `json:"user_info"`
219  }
220
221  type GetUserPrivacyResp {
222      Success bool `json:"success"`
223      HideMatchRecord bool `json:"hide_match_record"`
224  }
225
226  type UpdateUserPrivacyReq {
227      HideMatchRecord bool `json:"hide_match_record"`
228  }
229
230  type UpdateUserPrivacyResp {
231      Success bool `json:"success"`
232      Message string `json:"message"`
233      HideMatchRecord bool `json:"hide_match_record"`
234  }
235
236  type FavoriteVenueRewardStatusResp {
237      Success bool `json:"success"`
238      Enabled bool `json:"enabled"`
239      PopupEnabled bool `json:"popup_enabled"`
240      RewardDays int `json:"reward_days"`
241      NewUserWindowDays int `json:"new_user_window_days"`
242      Status string `json:"status"`
243      MemberExpiresAt string `json:"member_expires_at,optional"`
244      SubmittedVenueId int64 `json:"submitted_venue_id,optional"`
245      SubmittedVenueName string `json:"submitted_venue_name,optional"`
246      RejectReason string `json:"reject_reason,optional"`
247  }
248
249  // 更新推送令牌
250  type UpdatePushTokenReq {

```

```

251     PushClientId string `json:"push_client_id"
252 }
253
254 // 用户统计数据
255 type UserStatsResp {
256     Success    bool   `json:"success"
257     TotalMatches int   `json:"total_matches" // 总对局数
258     Wins       int    `json:"wins" // 胜场数
259     Losses     int    `json:"losses" // 负场数
260     WinRate    float64 `json:"win_rate" // 胜率(百分比)
261     MaxWinStreak int   `json:"max_win_streak" // 最高连胜
262 }
263
264 @server (
265     prefix: /api/user
266     group: user
267     jwt: Auth
268 )
269 service chasing_points-api {
270     @doc "获取当前用户信息"
271     @handler GetUserInfo
272     get /info returns (GetUserInfoResp)
273
274     @doc "获取用户隐私设置"
275     @handler GetUserPrivacy
276     get /privacy returns (GetUserPrivacyResp)
277
278     @doc "更新用户隐私设置"
279     @handler UpdateUserPrivacy
280     post /privacy (UpdateUserPrivacyReq) returns (UpdateUserPrivacyResp)
281
282     @doc "获取常玩球馆奖励状态"
283     @handler GetFavoriteVenueRewardStatus
284     get /favorite-venue-reward-status returns (FavoriteVenueRewardStatusResp)
285
286     @doc "获取用户统计数据"
287     @handler GetUserStats
288     get /stats returns (UserStatsResp)
289
290     @doc "更新昵称"
291     @handler UpdateNickname
292     post /nickname (UpdateNicknameReq) returns (UpdateNicknameResp)
293
294     @doc "更新推送令牌"
295     @handler UpdatePushToken
296     post /push-token (UpdatePushTokenReq) returns (CommonResp)
297 }
298
299 // ===== 对局相关 =====
300 // 通用响应

```

```

301  type CommonResp {
302      Success bool `json:"success"`
303      Message string `json:"message"`
304  }
305
306  // ===== 会员订阅 =====
307  type MemberPlanInfo {
308      PlanCode string `json:"plan_code"`
309      PlanName string `json:"plan_name"`
310      PriceFen int `json:"price_fen"`
311      PriceYuan string `json:"price_yuan"`
312      DurationDays int `json:"duration_days"`
313      DurationLabel string `json:"duration_label"`
314      Description string `json:"description"`
315      Highlight string `json:"highlight,optional"`
316  }
317
318  type GetMemberPlansResp {
319      Success bool `json:"success"`
320      CurrentTime string `json:"current_time"`
321      Plans []MemberPlanInfo `json:"plans"`
322  }
323
324  type GetMemberStatusResp {
325      Success bool `json:"success"`
326      IsActive bool `json:"is_active"`
327      CurrentTime string `json:"current_time"`
328      PlanCode string `json:"plan_code,optional"`
329      PlanName string `json:"plan_name,optional"`
330      MemberExpiresAt string `json:"member_expires_at,optional"`
331  }
332
333  type CreateMemberSubscriptionOrderReq {
334      PlanCode string `json:"plan_code"`
335      PayChannel string `json:"pay_channel"`
336  }
337
338  type WechatAppPayParams {
339      Appid string `json:"appid"`
340      Partnerid string `json:"partnerid"`
341      Prepayid string `json:"prepayid"`
342      Package string `json:"package"`
343      Noncestr string `json:"noncestr"`
344      Timestamp string `json:"timestamp"`
345      Sign string `json:"sign"`
346  }
347
348  type CreateMemberSubscriptionOrderResp {
349      Success bool `json:"success"`
350      Message string `json:"message"`

```

```

351   OrderNo      string    `json:"order_no"
352   PayChannel   string    `json:"pay_channel"
353   PlanCode     string    `json:"plan_code"
354   PlanName     string    `json:"plan_name"
355   AmountFen    int       `json:"amount_fen"
356   AmountYuan   string    `json:"amount_yuan"
357   Status       string    `json:"status"
358   AlipayOrderString string  `json:"alipay_order_string,optional"
359   WechatAppPayParams *WechatAppPayParams `json:"wechat_app_pay_params,optional"
360 }
361
362 type GetMemberSubscriptionOrderStatusReq {
363   OrderNo string `form:"order_no"
364 }
365
366 type GetMemberSubscriptionOrderStatusResp {
367   Success      bool `json:"success"
368   Message      string `json:"message,optional"
369   OrderNo      string `json:"order_no"
370   PayChannel   string `json:"pay_channel"
371   PlanCode     string `json:"plan_code"
372   PlanName     string `json:"plan_name"
373   AmountFen    int `json:"amount_fen"
374   AmountYuan   string `json:"amount_yuan"
375   Status       string `json:"status"
376   PaidAt       string `json:"paid_at,optional"
377   MemberExpiresAtBefore string `json:"member_expires_at_before,optional"
378   MemberExpiresAtAfter string `json:"member_expires_at_after,optional"
379 }
380
381 @server (
382   prefix: /api/member
383   group: member
384   jwt: Auth
385 )
386 service chasing_points-api {
387   @doc "获取会员套餐列表"
388   @handler GetMemberPlans
389   get /plans returns (GetMemberPlansResp)
390
391   @doc "获取会员状态"
392   @handler GetMemberStatus
393   get /status returns (GetMemberStatusResp)
394
395   @doc "创建会员订阅订单"
396   @handler CreateMemberSubscriptionOrder
397   post /order (CreateMemberSubscriptionOrderReq) returns (CreateMemberSubscriptionOrderResp)
398
399   @doc "查询会员订阅订单状态"
400   @handler GetMemberSubscriptionOrderStatus

```

```

401     get /order/status (GetMemberSubscriptionOrderStatusReq) returns (GetMemberSubscriptionOrderStatusResp)
402 }
403
404 @server (
405     prefix: /api/payment
406     group: payment
407 )
408 service chasing_points-api {
409     @doc "支付宝会员支付异步回调"
410     @handler AlipayMemberSubscriptionNotify
411     post /alipay/notify returns (CommonResp)
412
413     @doc "微信会员支付异步回调"
414     @handler WechatMemberSubscriptionNotify
415     post /wechat/notify returns (CommonResp)
416 }
417
418 // 特殊成绩项
419 type AchievementItem {
420     Type string `json:"type"`
421     Count int `json:"count"`
422 }
423
424 // 对局同步快照（当前登录用户视角）
425 type MatchSyncSnapshot {
426     MatchId          int64 `json:"match_id"`
427     Status            int `json:"status"`
428     ServerRevision    int64 `json:"server_revision"`
429     MyScore           int `json:"my_score"`
430     OpponentScore     int `json:"opponent_score"`
431     CurrentFrameStarted bool `json:"current_frame_started,optional"`
432     CurrentFrameMyScore int `json:"current_frame_my_score,optional"`
433     CurrentFrameOpponentScore int `json:"current_frame_opponent_score,optional"`
434     CurrentRound      int `json:"current_round"`
435     TotalRounds       int `json:"total_rounds"`
436     RedBallCount      int `json:"red_ball_count,optional"`
437     SnookerClearanceStarted bool `json:"snooker_clearance_started,optional"`
438     SnookerClearedColors []int `json:"snooker_cleared_colors,optional"`
439     SnookerExpectedClearanceScore int `json:"snooker_expected_clearance_score,optional"`
440     SnookerClearanceCompleted bool `json:"snooker_clearance_completed,optional"`
441 }
442
443 // 开始对局
444 type StartMatchReq {
445     GameType int `json:"game_type" // 1=斯诺克 2=九球追分 3=中式八球 4=美式九球
446     GameMode string `json:"game_mode,optional" // 比赛模式
447     OpponentId int64 `json:"opponent_id,optional" // 对手ID
448     OpponentName string `json:"opponent_name" // 对手昵称
449     OpponentAvatar string `json:"opponent_avatar,optional" // 对手头像
450 }
    
```

```

451
452 type StartMatchResp {
453     Success    bool        `json:"success"`
454     Action     string      `json:"action,optional" // created/resume_existing/blocked`
455     BlockReason string    `json:"block_reason,optional" // self_ongoing/opponent_ongoing`
456     Message    string      `json:"message,optional"`
457     MatchId    int64        `json:"match_id"`
458     OngoingMatch *CurrentMatchInfo `json:"ongoing_match,optional"`
459 }
460
461 // 加分
462 type MatchScoreReq {
463     MatchId    int64 `json:"match_id"`
464     Actor      int   `json:"actor" // 1=我 2=对手`
465     Score      int   `json:"score" // 加的分`
466     ClientActionId string `json:"client_action_id"`
467     BaseRevision int64 `json:"base_revision"`
468 }
469
470 type MatchScoreResp {
471     Accepted      bool        `json:"accepted"`
472     Success       bool        `json:"success"`
473     Message       string      `json:"message,optional"`
474     ClientActionId string    `json:"client_action_id,optional"`
475     ServerRevision int64      `json:"server_revision"`
476     Snapshot      MatchSyncSnapshot `json:"snapshot"`
477     RedBallCount  int         `json:"red_ball_count,optional"`
478     SnookerClearanceStarted bool    `json:"snooker_clearance_started,optional"`
479     SnookerClearedColors []int    `json:"snooker_cleared_colors,optional"`
480     SnookerExpectedClearanceScore int    `json:"snooker_expected_clearance_score,optional"`
481     SnookerClearanceCompleted bool    `json:"snooker_clearance_completed,optional"`
482     CurrentFrameStarted bool    `json:"current_frame_started,optional"`
483     CurrentFrameMyScore int      `json:"current_frame_my_score,optional"`
484     CurrentFrameOpponentScore int    `json:"current_frame_opponent_score,optional"`
485     MyScore       int         `json:"my_score"`
486     OpponentScore int         `json:"opponent_score"`
487 }
488
489 // 结束一局
490 type EndRoundReq {
491     MatchId    int64 `json:"match_id"`
492     Winner     int   `json:"winner" // 1=我 2=对手`
493     WinType    string `json:"win_type,optional" // 获胜方式`
494     Score      int   `json:"score,optional,default=1" // 获胜得分，默认1分`
495     ClientActionId string `json:"client_action_id"`
496     BaseRevision int64 `json:"base_revision"`
497 }
498
499 type EndRoundResp {
500     Accepted      bool        `json:"accepted"`
    
```

```

501     Success          bool          `json:"success"
502     ClientActionId    string        `json:"client_action_id,optional"
503     ServerRevision    int64         `json:"server_revision"
504     Snapshot           MatchSyncSnapshot `json:"snapshot"
505     RoundNo           int           `json:"round_no"
506     CurrentFrameStarted bool         `json:"current_frame_started,optional"
507     CurrentFrameMyScore int         `json:"current_frame_my_score,optional"
508     CurrentFrameOpponentScore int       `json:"current_frame_opponent_score,optional"
509     MyScore            int           `json:"my_score"
510     OpponentScore      int           `json:"opponent_score"
511 }
512
513 // 开始下一局
514 type StartNextRoundReq {
515     MatchId    int64 `json:"match_id"
516     ClientActionId string `json:"client_action_id"
517     BaseRevision int64 `json:"base_revision"
518 }
519
520 type StartNextRoundResp {
521     Accepted      bool          `json:"accepted"
522     Success       bool          `json:"success"
523     ClientActionId string        `json:"client_action_id,optional"
524     ServerRevision int64         `json:"server_revision"
525     Snapshot       MatchSyncSnapshot `json:"snapshot"
526     RoundNo        int           `json:"round_no" // 新局数
527     CurrentFrameStarted bool         `json:"current_frame_started,optional"
528     CurrentFrameMyScore int         `json:"current_frame_my_score,optional"
529     CurrentFrameOpponentScore int       `json:"current_frame_opponent_score,optional"
530     MyScore        int           `json:"my_score" // 我方总分
531     OpponentScore  int           `json:"opponent_score" // 对手总分
532 }
533
534 // 犯规
535 type MatchFoulReq {
536     MatchId    int64 `json:"match_id"
537     Actor      int   `json:"actor" // 犯规方
538     Score      int   `json:"score,optional" // 犯规加分，默认1
539     ClientActionId string `json:"client_action_id"
540     BaseRevision int64 `json:"base_revision"
541 }
542
543 // 撤销
544 type MatchUndoReq {
545     MatchId    int64 `json:"match_id"
546     ClientActionId string `json:"client_action_id"
547     BaseRevision int64 `json:"base_revision"
548 }
549
550 type MatchUndoResp {

```

```

551 Accepted      bool      `json:"accepted"
552 Success       bool      `json:"success"
553 ClientActionId string    `json:"client_action_id,optional"
554 ServerRevision int64     `json:"server_revision"
555 Snapshot       MatchSyncSnapshot `json:"snapshot"
556 CurrentFrameStarted bool    `json:"current_frame_started,optional"
557 CurrentFrameMyScore int      `json:"current_frame_my_score,optional"
558 CurrentFrameOpponentScore int   `json:"current_frame_opponent_score,optional"
559 MyScore        int        `json:"my_score"
560 OpponentScore   int        `json:"opponent_score"
561 Message        string     `json:"message"
562 }
563
564 // 结束对局
565 type FinishMatchReq {
566     MatchId    int64 `json:"match_id"
567     Remark     string `json:"remark,optional"
568     ClientActionId string `json:"client_action_id"
569     BaseRevision int64 `json:"base_revision"
570 }
571
572 type FinishMatchResp {
573     Accepted    bool    `json:"accepted"
574     Success     bool    `json:"success"
575     Result      int      `json:"result" // 1=胜利 2=失败 3=平局
576     ClientActionId string `json:"client_action_id,optional"
577     ServerRevision int64   `json:"server_revision"
578     Snapshot    MatchSyncSnapshot `json:"snapshot"
579     MyScore     int       `json:"my_score"
580     OpponentScore int      `json:"opponent_score"
581 }
582
583 // 取消对局
584 type CancelMatchReq {
585     MatchId int64 `json:"match_id"
586 }
587
588 // 获取进行中局
589 type CurrentMatchInfo {
590     Id          int64 `json:"id"
591     GameType     int   `json:"game_type"
592     GameTypeName string `json:"game_type_name"
593     GameMode     string `json:"game_mode"
594     OpponentId   int64 `json:"opponent_id,optional"
595     OpponentName string `json:"opponent_name"
596     OpponentAvatar string `json:"opponent_avatar,optional"
597     MyScore      int   `json:"my_score"
598     OpponentScore int   `json:"opponent_score"
599     CurrentFrameStarted bool `json:"current_frame_started,optional"
600     CurrentFrameMyScore int `json:"current_frame_my_score,optional"

```

```

601   CurrentFrameOpponentScore int `json:"current_frame_opponent_score,optional"
602   CurrentRound      int `json:"current_round"
603   ServerRevision    int64 `json:"server_revision"
604   MatchTime         string `json:"match_time"
605   DurationSeconds   int64 `json:"duration_seconds" // 对局已持续秒数
606 }
607
608 type GetCurrentMatchResp {
609   Success bool `json:"success"
610   Match *CurrentMatchInfo `json:"match"
611 }
612
613 // 对局列表
614 type MatchListReq {
615   Page int `form:"page,default=1"
616   PageSize int `form:"page_size,default=20"
617   GameType int `form:"game_type,optional"
618   Result int `form:"result,optional"
619 }
620
621 type MatchListItem {
622   Id int64 `json:"id"
623   OpponentId int64 `json:"opponent_id" // 对手ID（注册用户才有）
624   GameType int `json:"game_type"
625   GameTypeName string `json:"game_type_name"
626   OpponentName string `json:"opponent_name"
627   OpponentAvatar string `json:"opponent_avatar"
628   MyScore int `json:"my_score"
629   OpponentScore int `json:"opponent_score"
630   Result int `json:"result"
631   MatchTime string `json:"match_time"
632 }
633
634 type MatchListResp {
635   Success bool `json:"success"
636   Total int64 `json:"total"
637   List []MatchListItem `json:"list"
638 }
639
640 // 交锋统计
641 type H2HStatsReq {
642   TargetUserId int64 `form:"target_user_id,optional"
643   OpponentId int64 `form:"opponent_id,optional"
644   OpponentName string `form:"opponent_name,optional"
645 }
646
647 type H2HOpponent {
648   Id int64 `json:"id"
649   Name string `json:"name"
650   Avatar string `json:"avatar"

```

```

651 }
652
653 type H2HStats {
654     TotalMatches int `json:"total_matches"`
655     MyWins int `json:"my_wins"`
656     OpponentWins int `json:"opponent_wins"`
657     WinRate float64 `json:"win_rate"`
658     AvgScoreDiff float64 `json:"avg_score_diff"`
659     MaxWinStreak int `json:"max_win_streak"`
660 }
661
662 type H2HStatsResp {
663     Success bool `json:"success"`
664     Message string `json:"message,optional"`
665     Opponent *H2HOpponent `json:"opponent"`
666     Stats *H2HStats `json:"stats"`
667 }
668
669 // 交锋历史
670 type H2HHistoryReq {
671     TargetUserId int64 `form:"target_user_id,optional"`
672     OpponentId int64 `form:"opponent_id,optional"`
673     OpponentName string `form:"opponent_name,optional"`
674     Page int `form:"page,default=1"`
675     PageSize int `form:"page_size,default=20"`
676     Result int `form:"result,optional"`
677 }
678
679 type H2HHistoryResp {
680     Success bool `json:"success"`
681     Message string `json:"message,optional"`
682     Total int64 `json:"total"`
683     List []MatchListItem `json:"list"`
684 }
685
686 // 搜索对手
687 type SearchOpponentReq {
688     Keyword string `form:"keyword"`
689 }
690
691 type OpponentItem {
692     Id int64 `json:"id"`
693     Name string `json:"name"`
694     Avatar string `json:"avatar"`
695     MatchCount int `json:"match_count"`
696 }
697
698 type SearchOpponentResp {
699     Success bool `json:"success"`
700     List []OpponentItem `json:"list"`

```

```

701  }
702
703  // 获取匹配二维码
704  type GetMatchQRCodeResp {
705      Success bool `json:"success"`
706      QrcodeData string `json:"qrcode_data"` // 二维码内容JSON: {"user_id":xxx,"nickname":"xxx","avatar":"xxx","ts":xxx}
707  }
708
709  // 获取对局详情
710  type GetMatchDetailReq {
711      MatchId int64 `form:"match_id"`
712  }
713
714  type MatchAchievement {
715      BreakClear int `json:"break_clear"` // 炸清次数
716      ContinueClear int `json:"continue_clear"` // 接清次数
717      GoldenBreak int `json:"golden_break"` // 金球数
718      NineOnBreak int `json:"nine_on_break"` // 大金数
719      Break50 int `json:"break_50"` // 单杆50+
720      Break100 int `json:"break_100"` // 单杆100+
721      Break147 int `json:"break_147"` // 单杆147
722  }
723
724  type RankDetail {
725      Label string `json:"label"`
726      Value int `json:"value"`
727  }
728
729  type GetMatchDetailResp {
730      Success bool `json:"success"`
731      Match MatchDetailData `json:"match"`
732  }
733
734  type MatchSummaryItem {
735      Label string `json:"label"`
736      Value string `json:"value"`
737      Desc string `json:"desc,optional"`
738      Icon string `json:"icon,optional"`
739      Tone string `json:"tone,optional"`
740  }
741
742  type MatchDetailData {
743      Id int64 `json:"id"`
744      GameType int `json:"game_type"`
745      Status int `json:"status"` // 1=进行中 2=已完成 3=已取消
746      ServerRevision int64 `json:"server_revision"`
747      IsPlayer1 bool `json:"is_player1"` // 当前用户是否是创建者(用于视角判断)
748      MyScore int `json:"my_score"` // 从当前用户视角的分数
749      OpponentScore int `json:"opponent_score"` // 从当前用户视角的对手分数
750      MyName string `json:"my_name"`

```

```

751   OpponentName      string      `json:"opponent_name"
752   MyAvatar           string      `json:"my_avatar"
753   OpponentAvatar      string      `json:"opponent_avatar"
754   MyWinRate           float64     `json:"my_win_rate" // 我方胜率(百分比)
755   OpponentWinRate      float64     `json:"opponent_win_rate" // 对手胜率(百分比)
756   MyMaxScore          int         `json:"my_max_score" // 我方单杆最高
757   OpponentMaxScore     int         `json:"opponent_max_score" // 对手单杆最高
758   MyRankChange        int         `json:"my_rank_change"
759   OpponentRankChange   int         `json:"opponent_rank_change"
760   MyRankDetails        []RankDetail `json:"my_rank_details"
761   CurrentFrameStarted  bool        `json:"current_frame_started,optional"
762   CurrentFrameMyScore  int         `json:"current_frame_my_score,optional"
763   CurrentFrameOpponentScore int       `json:"current_frame_opponent_score,optional"
764   RedBallCount         int         `json:"red_ball_count,optional"
765   SnookerClearanceStarted bool       `json:"snooker_clearance_started,optional"
766   SnookerClearedColors []int       `json:"snooker_cleared_colors,optional"
767   SnookerExpectedClearanceScore int       `json:"snooker_expected_clearance_score,optional"
768   SnookerClearanceCompleted bool       `json:"snooker_clearance_completed,optional"
769   SummaryHighlights     []MatchSummaryItem `json:"summary_highlights,optional"
770   SummaryStats          []MatchSummaryItem `json:"summary_stats,optional"
771   Achievements          MatchAchievement `json:"achievements"
772   CreatedAt             string      `json:"created_at"
773 }
774
775 @server (
776   prefix: /api/match
777   group: match
778   jwt: Auth
779 )
780 service chasing_points-api {
781   @doc "开始对局"
782   @handler StartMatch
783   post /start (StartMatchReq) returns (StartMatchResp)
784
785   @doc "加分"
786   @handler MatchScore
787   post /score (MatchScoreReq) returns (MatchScoreResp)
788
789   @doc "结束一局"
790   @handler EndRound
791   post /round/end (EndRoundReq) returns (EndRoundResp)
792
793   @doc "开始下一局"
794   @handler StartNextRound
795   post /round/start (StartNextRoundReq) returns (StartNextRoundResp)
796
797   @doc "犯规"
798   @handler MatchFoul
799   post /foul (MatchFoulReq) returns (MatchScoreResp)
800
    
```

```

801  @doc "撤销操作"
802  @handler MatchUndo
803  post /undo (MatchUndoReq) returns (MatchUndoResp)
804
805  @doc "结束对局"
806  @handler FinishMatch
807  post /finish (FinishMatchReq) returns (FinishMatchResp)
808
809  @doc "取消对局"
810  @handler CancelMatch
811  post /cancel (CancelMatchReq) returns (CommonResp)
812
813  @doc "获取进行中对局"
814  @handler GetCurrentMatch
815  get /current returns (GetCurrentMatchResp)
816
817  @doc "获取对局列表"
818  @handler GetMatchList
819  get /list (MatchListReq) returns (MatchListResp)
820
821  @doc "获取交锋统计"
822  @handler GetH2HStats
823  get /h2h/stats (H2HStatsReq) returns (H2HStatsResp)
824
825  @doc "获取交锋历史"
826  @handler GetH2HHistory
827  get /h2h/history (H2HHistoryReq) returns (H2HHistoryResp)
828
829  @doc "获取匹配二维码"
830  @handler GetMatchQRCode
831  get /qrcode returns (GetMatchQRCodeResp)
832
833  @doc "获取对局详情"
834  @handler GetMatchDetail
835  get /detail (GetMatchDetailReq) returns (GetMatchDetailResp)
836  }
837
838  // 对手记录列表项
839  type OpponentRecordItem {
840    Id      int64 `json:"id"`
841    Name    string `json:"name"`
842    Avatar  string `json:"avatar"`
843    TotalMatches int `json:"total_matches" // 总对局数
844    Wins    int `json:"wins" // 我方胜场
845    Losses  int `json:"losses" // 我方负场
846    WinRate float64 `json:"win_rate" // 胜率百分比
847    LastMatchAt string `json:"last_match_at" // 上次对局时间
848  }
849
850  // 获取对手列表请求

```

```

851 type GetOpponentListReq {
852     Page    int `form:"page,default=1"
853     PageSize int `form:"page_size,default=20"
854     Keyword  string `form:"keyword,optional"
855     TargetUserId int64 `form:"target_user_id,optional"
856 }
857
858 // 获取对手列表响应
859 type GetOpponentListResp {
860     Success    bool    `json:"success"
861     Message    string  `json:"message,optional"
862     Hidden     bool    `json:"hidden"
863     TotalOpponents int  `json:"total_opponents" // 总对手数
864     TotalWins  int     `json:"total_wins" // 总胜场
865     Total     int64    `json:"total"
866     List      []OpponentRecordItem `json:"list"
867 }
868
869 @server (
870     prefix: /api/opponent
871     group: opponent
872     jwt: Auth
873 )
874 service chasing_points-api {
875     @doc "搜索对手"
876     @handler SearchOpponent
877     get /search (SearchOpponentReq) returns (SearchOpponentResp)
878
879     @doc (
880         summary: "获取对手列表"
881         description: "获取当前用户交战过的所有对手列表及统计数据"
882     )
883     @handler GetOpponentList
884     get /list (GetOpponentListReq) returns (GetOpponentListResp)
885 }
886
887 // ===== 段位相关 =====
888 // 段位信息
889 type RankInfo {
890     Level  int `json:"level" // 段位等级 1-5
891     Name   string `json:"name" // 段位名称
892     Icon   string `json:"icon" // 段位图标URL
893     RankScore int `json:"rank_score" // 当前排位分
894     TotalWins int `json:"total_wins" // 总胜场
895     TotalLosses int `json:"total_losses" // 总败场
896     MaxStreak int `json:"max_streak" // 最高连胜
897     NextLevel int `json:"next_level" // 下一段位等级
898     NextName string `json:"next_name" // 下一段位名称
899     NextScore int `json:"next_score" // 晋级所需分数
900     Progress int `json:"progress" // 晋级进度百分比
    
```

```

901  }
902
903  // 获取用户段位信息
904  type GetUserRankInfoReq {
905      GameType int `form:"game_type,optional,default=3"`
906  }
907
908  type GetUserRankInfoResp {
909      Success bool `json:"success"`
910      RankInfo *RankInfo `json:"rank_info"`
911  }
912
913  // 段位列表项
914  type RankItem {
915      Level int `json:"level"`
916      Name string `json:"name"`
917      Icon string `json:"icon"`
918      MinScore int `json:"min_score" // 晋升最低分`
919      IsCurrent bool `json:"is_current" // 是否当前段位`
920  }
921
922  // 获取段位列表
923  type GetRankListReq {
924      GameType int `form:"game_type,optional,default=3"`
925  }
926
927  type GetRankListResp {
928      Success bool `json:"success"`
929      List []RankItem `json:"list"`
930  }
931
932  // 排行榜用户项
933  type LeaderboardItem {
934      Rank int `json:"rank" // 排名`
935      UserId int64 `json:"user_id"`
936      Nickname string `json:"nickname"`
937      Avatar string `json:"avatar"`
938      RankLevel int `json:"rank_level" // 段位等级`
939      RankName string `json:"rank_name" // 段位名称`
940      RankScore int `json:"rank_score" // 排位分`
941      WinRate int `json:"win_rate" // 胜率百分比`
942  }
943
944  // 获取排行榜请求
945  type GetLeaderboardReq {
946      GameType int `form:"game_type,optional,default=3"`
947      Page int `form:"page,default=1"`
948      PageSize int `form:"page_size,default=20"`
949  }
950

```

```

951 // 获取排行榜响应
952 type GetLeaderboardResp {
953     Success bool    `json:"success"`
954     Total  int64    `json:"total"`
955     TopThree []LeaderboardItem `json:"top_three" // 前三名
956     MyRanking *LeaderboardItem `json:"my_ranking" // 我的排名
957     List []LeaderboardItem `json:"list" // 第4名开始的列表
958 }
959
960 @server (
961     prefix: /api/rank
962     group: rank
963     jwt: Auth
964 )
965 service chasing_points-api {
966     @doc (
967         summary: "获取用户段位信息"
968         description: "获取当前登录用户的段位信息，包括段位等级、排位分和晋级进度"
969     )
970     @handler GetUserRankInfo
971     get /info (GetUserRankInfoReq) returns (GetUserRankInfoResp)
972
973     @doc (
974         summary: "获取段位列表"
975         description: "获取所有段位列表及其晋升条件"
976     )
977     @handler GetRankList
978     get /list (GetRankListReq) returns (GetRankListResp)
979 }
980
981 // ===== 成就模块 =====
982 // 成就定义
983 type AchievementDef {
984     Id int64 `json:"id"`
985     Key string `json:"key"`
986     Name string `json:"name"`
987     Description string `json:"description"`
988     Icon string `json:"icon"`
989     Category string `json:"category"`
990     Threshold int `json:"threshold"`
991     Progress int `json:"progress"`
992     Unlocked bool `json:"unlocked"`
993     UnlockedAt string `json:"unlocked_at,omitempty"`
994 }
995
996 type GetAchievementListReq {
997     Category string `form:"category,optional"`
998 }
999
1000 type GetAchievementListResp {

```

```

1001     Success bool      `json:"success"
1002     List  []AchievementDef `json:"list"
1003 }
1004
1005 type GetUserAchievementsResp {
1006     Success bool      `json:"success"
1007     Total  int        `json:"total"
1008     List  []AchievementDef `json:"list"
1009 }
1010
1011 // 称号
1012 type TitleInfo {
1013     Id    int64 `json:"id"
1014     TitleName string `json:"title_name"
1015     Source string `json:"source"
1016     Equipped bool `json:"equipped"
1017     CreatedAt string `json:"created_at"
1018 }
1019
1020 type EquipTitleReq {
1021     TitleId int64 `json:"title_id"
1022     Equip bool `json:"equip"
1023 }
1024
1025 type GetUserTitlesResp {
1026     Success bool      `json:"success"
1027     List  []TitleInfo `json:"list"
1028 }
1029
1030 @server (
1031     prefix: /api/achievement
1032     group: achievement
1033     jwt: Auth
1034 )
1035 service chasing_points-api {
1036     @doc "获取成就列表"
1037     @handler GetAchievementList
1038     get /list (GetAchievementListReq) returns (GetAchievementListResp)
1039
1040     @doc "获取已解锁成就"
1041     @handler GetUserAchievements
1042     get /unlocked returns (GetUserAchievementsResp)
1043
1044     @doc "装备/卸下称号"
1045     @handler EquipTitle
1046     post /title/equip (EquipTitleReq) returns (CommonResp)
1047
1048     @doc "获取用户称号列表"
1049     @handler GetUserTitles
1050     get /titles returns (GetUserTitlesResp)

```

```

1051 }
1052
1053 // ===== 好友模块 =====
1054 type FriendInfo {
1055     Id    int64 `json:"id"`
1056     UserId int64 `json:"user_id"`
1057     Nickname string `json:"nickname"`
1058     Avatar  string `json:"avatar"`
1059     RankLevel int `json:"rank_level"`
1060     RankName string `json:"rank_name"`
1061     CreatedAt string `json:"created_at"`
1062 }
1063
1064 type FriendRequestInfo {
1065     Id    int64 `json:"id"`
1066     FromUserId int64 `json:"from_user_id"`
1067     Nickname string `json:"nickname"`
1068     Avatar  string `json:"avatar"`
1069     Message string `json:"message"`
1070     Status  int `json:"status"`
1071     CreatedAt string `json:"created_at"`
1072 }
1073
1074 type SendFriendRequestReq {
1075     ToUserId int64 `json:"to_user_id"`
1076     Message string `json:"message,optional"`
1077 }
1078
1079 type HandleFriendRequestReq {
1080     RequestId int64 `json:"request_id"`
1081 }
1082
1083 type GetFriendListReq {
1084     Page  int `form:"page,default=1"`
1085     PageSize int `form:"page_size,default=20"`
1086     Keyword string `form:"keyword,optional"`
1087 }
1088
1089 type GetFriendListResp {
1090     Success bool `json:"success"`
1091     Total int64 `json:"total"`
1092     List []FriendInfo `json:"list"`
1093 }
1094
1095 type DeleteFriendReq {
1096     FriendUserId int64 `json:"friend_user_id"`
1097 }
1098
1099 type BlacklistFriendReq {
1100     FriendUserId int64 `json:"friend_user_id"`

```

```

1101 }
1102
1103 type GetFriendRequestsReq {
1104     Page    int `form:"page,default=1"`
1105     PageSize int `form:"page_size,default=20"`
1106 }
1107
1108 type GetFriendRequestsResp {
1109     Success bool    `json:"success"`
1110     Total   int64    `json:"total"`
1111     List    []FriendRequestInfo `json:"list"`
1112 }
1113
1114 type SearchUserReq {
1115     Keyword string `form:"keyword"`
1116 }
1117
1118 type SearchUserItem {
1119     UserId int64 `json:"user_id"`
1120     Nickname string `json:"nickname"`
1121     Avatar string `json:"avatar"`
1122     RankName string `json:"rank_name"`
1123     IsFriend bool `json:"is_friend"`
1124 }
1125
1126 type SearchUserResp {
1127     Success bool    `json:"success"`
1128     List    []SearchUserItem `json:"list"`
1129 }
1130
1131 @server (
1132     prefix: /api/friend
1133     group: friend
1134     jwt: Auth
1135 )
1136 service chasing_points-api {
1137     @doc "发送好友请求"
1138     @handler SendFriendRequest
1139     post /request (SendFriendRequestReq) returns (CommonResp)
1140
1141     @doc "接受好友请求"
1142     @handler AcceptFriendRequest
1143     post /accept (HandleFriendRequestReq) returns (CommonResp)
1144
1145     @doc "拒绝好友请求"
1146     @handler RejectFriendRequest
1147     post /reject (HandleFriendRequestReq) returns (CommonResp)
1148
1149     @doc "获取好友列表"
1150     @handler GetFriendList

```

```

1151     get /list (GetFriendListReq) returns (GetFriendListResp)
1152
1153     @doc "删除好友"
1154     @handler DeleteFriend
1155     post /delete (DeleteFriendReq) returns (CommonResp)
1156
1157     @doc "加入黑名单"
1158     @handler BlacklistFriend
1159     post /blacklist (BlacklistFriendReq) returns (CommonResp)
1160
1161     @doc "获取好友请求列表"
1162     @handler GetFriendRequests
1163     get /requests (GetFriendRequestsReq) returns (GetFriendRequestsResp)
1164
1165     @doc "搜索用户"
1166     @handler SearchFriendUser
1167     get /search (SearchUserReq) returns (SearchUserResp)
1168 }
1169
1170 // ===== 关注模块 =====
1171 type FollowUserInfo {
1172     UserId    int64 `json:"user_id"`
1173     Nickname  string `json:"nickname"`
1174     Avatar    string `json:"avatar"`
1175     RankName  string `json:"rank_name"`
1176     IsFollowing bool `json:"is_following"`
1177 }
1178
1179 type FollowReq {
1180     TargetUserId int64 `json:"target_user_id"`
1181 }
1182
1183 type GetFollowListReq {
1184     Page    int `form:"page,default=1"`
1185     PageSize int `form:"page_size,default=20"`
1186 }
1187
1188 type GetFollowListResp {
1189     Success bool    `json:"success"`
1190     Total   int64    `json:"total"`
1191     List    []FollowUserInfo `json:"list"`
1192 }
1193
1194 @server (
1195     prefix: /api/follow
1196     group: follow
1197     jwt: Auth
1198 )
1199 service chasing_points-api {
1200     @doc "关注用户"

```

```

1201  @handler FollowUser
1202  post /follow (FollowReq) returns (CommonResp)
1203
1204  @doc "取消关注"
1205  @handler UnfollowUser
1206  post /unfollow (FollowReq) returns (CommonResp)
1207
1208  @doc "获取关注列表"
1209  @handler GetFollowingList
1210  get /following (GetFollowListReq) returns (GetFollowListResp)
1211
1212  @doc "获取粉丝列表"
1213  @handler GetFollowerList
1214  get /followers (GetFollowListReq) returns (GetFollowListResp)
1215  }
1216
1217  // ===== 动态模块 =====
1218  type SocialPostInfo {
1219      Id      int64 `json:"id"`
1220      UserId  int64 `json:"user_id"`
1221      Nickname string `json:"nickname"`
1222      Avatar  string `json:"avatar"`
1223      Content string `json:"content"`
1224      Images  []string `json:"images"`
1225      PostType int `json:"post_type"`
1226      MatchId int64 `json:"match_id,omitempty"`
1227      LikesCount int `json:"likes_count"`
1228      CommentsCount int `json:"comments_count"`
1229      IsLiked bool `json:"is_liked"`
1230      Status int `json:"status"`
1231      StatusText string `json:"status_text"`
1232      RejectReason string `json:"reject_reason,optional"`
1233      IsMine bool `json:"is_mine"`
1234      CreatedAt string `json:"created_at"`
1235  }
1236
1237  type CreatePostReq {
1238      Content string `json:"content"`
1239      Images []string `json:"images,optional"`
1240      PostType int `json:"post_type"`
1241      MatchId int64 `json:"match_id,optional"`
1242  }
1243
1244  type CreatePostResp {
1245      Success bool `json:"success"`
1246      PostId int64 `json:"post_id"`
1247  }
1248
1249  type GetPostListReq {
1250      Page int `form:"page,default=1"`

```

```

1251     PageSize int `form:"page_size,default=20"
1252 }
1253
1254 type GetPostListResp {
1255     Success bool      `json:"success"
1256     Total  int64      `json:"total"
1257     List  []SocialPostInfo `json:"list"
1258 }
1259
1260 type PostIdReq {
1261     PostId int64 `json:"post_id"
1262 }
1263
1264 type CommentPostReq {
1265     PostId int64 `json:"post_id"
1266     Content string `json:"content"
1267 }
1268
1269 type PostCommentInfo {
1270     Id int64 `json:"id"
1271     UserId int64 `json:"user_id"
1272     Nickname string `json:"nickname"
1273     Avatar string `json:"avatar"
1274     Content string `json:"content"
1275     CreatedAt string `json:"created_at"
1276 }
1277
1278 type GetPostCommentsReq {
1279     PostId int64 `form:"post_id"
1280     Page int `form:"page,default=1"
1281     PageSize int `form:"page_size,default=20"
1282 }
1283
1284 type GetPostCommentsResp {
1285     Success bool      `json:"success"
1286     Total int64      `json:"total"
1287     List []PostCommentInfo `json:"list"
1288 }
1289
1290 @server (
1291     prefix: /api/social
1292     group: social
1293 )
1294 service chasing_points-api {
1295     @doc "获取广场动态"
1296     @handler GetPublicPosts
1297     get /public (GetPostListReq) returns (GetPostListResp)
1298 }
1299
1300 @server (

```

```

1301     prefix: /api/social
1302     group: social
1303     jwt: Auth
1304 )
1305 service chasing_points-api {
1306     @doc "发布动态"
1307     @handler CreatePost
1308     post /post (CreatePostReq) returns (CreatePostResp)
1309
1310     @doc "获取关注的人的动态"
1311     @handler GetPostList
1312     get /feed (GetPostListReq) returns (GetPostListResp)
1313
1314     @doc "获取我的动态"
1315     @handler GetMyPostList
1316     get /mine (GetPostListReq) returns (GetPostListResp)
1317
1318     @doc "点赞动态"
1319     @handler LikePost
1320     post /like (PostIdReq) returns (CommonResp)
1321
1322     @doc "取消点赞"
1323     @handler UnlikePost
1324     post /unlike (PostIdReq) returns (CommonResp)
1325
1326     @doc "评论动态"
1327     @handler CommentPost
1328     post /comment (CommentPostReq) returns (CommonResp)
1329
1330     @doc "删除动态"
1331     @handler DeletePost
1332     post /delete (PostIdReq) returns (CommonResp)
1333
1334     @doc "获取动态评论"
1335     @handler GetPostComments
1336     get /comments (GetPostCommentsReq) returns (GetPostCommentsResp)
1337 }
1338
1339 // ===== 挑战模块 =====
1340 type ChallengeInfo {
1341     Id      int64 `json:"id"`
1342     FromUserId int64 `json:"from_user_id"`
1343     FromNickname string `json:"from_nickname"`
1344     FromAvatar string `json:"from_avatar"`
1345     ToUserId int64 `json:"to_user_id"`
1346     ToNickname string `json:"to_nickname"`
1347     ToAvatar string `json:"to_avatar"`
1348     GameType int `json:"game_type"`
1349     Message string `json:"message"`
1350     Status int `json:"status"`

```

```

1351     CreatedAt string `json:"created_at"
1352 }
1353
1354 type SendChallengeReq {
1355     ToUserId int64 `json:"to_user_id"
1356     GameType int `json:"game_type"
1357     Message string `json:"message,optional"
1358 }
1359
1360 type SendChallengeResp {
1361     Success bool `json:"success"
1362     ChallengeId int64 `json:"challenge_id"
1363 }
1364
1365 type HandleChallengeReq {
1366     ChallengeId int64 `json:"challenge_id"
1367 }
1368
1369 type GetPendingChallengesResp {
1370     Success bool `json:"success"
1371     List []ChallengeInfo `json:"list"
1372 }
1373
1374 @server (
1375     prefix: /api/challenge
1376     group: challenge
1377     jwt: Auth
1378 )
1379 service chasing_points-api {
1380     @doc "发起挑战"
1381     @handler SendChallenge
1382     post /send (SendChallengeReq) returns (SendChallengeResp)
1383
1384     @doc "接受挑战"
1385     @handler AcceptChallenge
1386     post /accept (HandleChallengeReq) returns (CommonResp)
1387
1388     @doc "拒绝挑战"
1389     @handler RejectChallenge
1390     post /reject (HandleChallengeReq) returns (CommonResp)
1391
1392     @doc "获取待处理挑战"
1393     @handler GetPendingChallenges
1394     get /pending returns (GetPendingChallengesResp)
1395 }
1396
1397 // ===== 赛事模块 =====
1398 type TournamentInfo {
1399     Id int64 `json:"id"
1400     CreatorId int64 `json:"creator_id"

```

```

1401   CreatorName string `json:"creator_name"
1402   Name         string `json:"name"
1403   Description  string `json:"description"
1404   GameType    int   `json:"game_type"
1405   Format      int   `json:"format"
1406   MaxPlayers  int   `json:"max_players"
1407   CurrentPlayers int `json:"current_players"
1408   Status      int   `json:"status"
1409   City        string `json:"city"
1410   VenueName   string `json:"venue_name"
1411   StartTime   string `json:"start_time"
1412   EndTime     string `json:"end_time"
1413   CreatedAt   string `json:"created_at"
1414 }
1415
1416 type GetTournamentListReq {
1417   Page int `form:"page,default=1"
1418   PageSize int `form:"page_size,default=20"
1419   Status int `form:"status,optional"
1420   GameType int `form:"game_type,optional"
1421   City string `form:"city,optional"
1422 }
1423
1424 type GetTournamentListResp {
1425   Success bool `json:"success"
1426   Total int64 `json:"total"
1427   List []TournamentInfo `json:"list"
1428 }
1429
1430 type GetTournamentDetailReq {
1431   TournamentId int64 `form:"tournament_id"
1432 }
1433
1434 type TournamentParticipant {
1435   UserId int64 `json:"user_id"
1436   Nickname string `json:"nickname"
1437   Avatar string `json:"avatar"
1438   Seed int `json:"seed"
1439   Status int `json:"status"
1440   FinalRank int `json:"final_rank"
1441 }
1442
1443 type GetTournamentDetailResp {
1444   Success bool `json:"success"
1445   Tournament *TournamentInfo `json:"tournament"
1446   Participants []TournamentParticipant `json:"participants"
1447 }
1448
1449 type TournamentBracketMatch {
1450   Id int64 `json:"id"

```

```

1451 RoundNumber int `json:"round_number"
1452 MatchOrder int `json:"match_order"
1453 Player1Id int64 `json:"player1_id"
1454 Player1Name string `json:"player1_name"
1455 Player2Id int64 `json:"player2_id"
1456 Player2Name string `json:"player2_name"
1457 WinnerId int64 `json:"winner_id"
1458 BracketPosition string `json:"bracket_position"
1459 Status int `json:"status"
1460 }
1461
1462 type GetTournamentBracketReq {
1463     TournamentId int64 `form:"tournament_id"
1464 }
1465
1466 type GetTournamentBracketResp {
1467     Success bool `json:"success"
1468     Matches []TournamentBracketMatch `json:"matches"
1469     TotalRounds int `json:"total_rounds"
1470 }
1471
1472 // ===== 赛事情报模块 =====
1473 type EventNewsInfo {
1474     Id int64 `json:"id"
1475     Title string `json:"title"
1476     GameType int `json:"game_type"
1477     SourceType string `json:"source_type"
1478     SourceName string `json:"source_name"
1479     SourceUrl string `json:"source_url"
1480     CoverImage string `json:"cover_image"
1481     Summary string `json:"summary"
1482     Content string `json:"content"
1483     Country string `json:"country"
1484     City string `json:"city"
1485     Venue string `json:"venue"
1486     StartTime string `json:"start_time"
1487     EndTime string `json:"end_time"
1488     Status int `json:"status" // 0:即将开始 1:进行中 2:已结束 3:已取消
1489     CurrentStageText string `json:"current_stage_text"
1490     LatestResultText string `json:"latest_result_text"
1491     StageCount int `json:"stage_count"
1492     Stages []EventNewsStageInfo `json:"stages"
1493     Featured bool `json:"featured"
1494     SortTime string `json:"sort_time"
1495     Published bool `json:"published"
1496     PublishedAt string `json:"published_at"
1497     CreatedAt string `json:"created_at"
1498     UpdatedAt string `json:"updated_at"
1499 }
1500

```

```

1501     OpponentScore    int        `json:"opponent_score"
1502 }
1503
1504 type EquipTitleReq struct {
1505     TitleId int64 `json:"title_id"
1506     Equip   bool  `json:"equip"
1507 }
1508
1509 type EventNewsInfo struct {
1510     Id      int64      `json:"id"
1511     Title   string     `json:"title"
1512     GameType int       `json:"game_type"
1513     SourceType string    `json:"source_type"
1514     SourceName string   `json:"source_name"
1515     SourceUrl string   `json:"source_url"
1516     CoverImage string  `json:"cover_image"
1517     Summary string   `json:"summary"
1518     Content string   `json:"content"
1519     Country string   `json:"country"
1520     City    string   `json:"city"
1521     Venue   string   `json:"venue"
1522     StartTime string  `json:"start_time"
1523     EndTime string  `json:"end_time"
1524     Status   int      `json:"status" // 0:即将开始 1:进行中 2:已结束 3:已取消
1525     CurrentStageText string `json:"current_stage_text"
1526     LatestResultText string `json:"latest_result_text"
1527     StageCount int       `json:"stage_count"
1528     Stages    []EventNewsStageInfo `json:"stages"
1529     Featured  bool       `json:"featured"
1530     SortTime  string    `json:"sort_time"
1531     Published bool     `json:"published"
1532     PublishedAt string   `json:"published_at"
1533     CreatedAt string   `json:"created_at"
1534     UpdatedAt string   `json:"updated_at"
1535 }
1536
1537 type EventNewsStageInfo struct {
1538     Id      int64 `json:"id"
1539     EventId int64 `json:"event_id"
1540     StageName string `json:"stage_name"
1541     StageOrder int   `json:"stage_order"
1542     StartTime string `json:"start_time"
1543     EndTime string `json:"end_time"
1544     Status int `json:"status"
1545     ResultText string `json:"result_text"
1546     SortTime string `json:"sort_time"
1547     CreatedAt string `json:"created_at"
1548     UpdatedAt string `json:"updated_at"
1549 }
1550

```

```

1551 type FavoriteVenueRewardStatusResp struct {
1552     Success    bool `json:"success"`
1553     Enabled     bool `json:"enabled"`
1554     PopupEnabled bool `json:"popup_enabled"`
1555     RewardDays  int  `json:"reward_days"`
1556     NewUserWindowDays int `json:"new_user_window_days"`
1557     Status      string `json:"status"`
1558     MemberExpiresAt string `json:"member_expires_at,optional"`
1559     SubmittedVenueId int64 `json:"submitted_venue_id,optional"`
1560     SubmittedVenueName string `json:"submitted_venue_name,optional"`
1561     RejectReason string `json:"reject_reason,optional"`
1562 }
1563
1564 type FinishMatchReq struct {
1565     MatchId    int64 `json:"match_id"`
1566     Remark     string `json:"remark,optional"`
1567     ClientActionId string `json:"client_action_id"`
1568     BaseRevision int64 `json:"base_revision"`
1569 }
1570
1571 type FinishMatchResp struct {
1572     Accepted    bool `json:"accepted"`
1573     Success     bool `json:"success"`
1574     Result      int  `json:"result" // 1=胜利 2=失败 3=平局`
1575     ClientActionId string `json:"client_action_id,optional"`
1576     ServerRevision int64 `json:"server_revision"`
1577     Snapshot    MatchSyncSnapshot `json:"snapshot"`
1578     MyScore     int  `json:"my_score"`
1579     OpponentScore int  `json:"opponent_score"`
1580 }
1581
1582 type FollowReq struct {
1583     TargetUserId int64 `json:"target_user_id"`
1584 }
1585
1586 type FollowUserInfo struct {
1587     UserId    int64 `json:"user_id"`
1588     Nickname  string `json:"nickname"`
1589     Avatar    string `json:"avatar"`
1590     RankName  string `json:"rank_name"`
1591     IsFollowing bool `json:"is_following"`
1592 }
1593
1594 type FriendInfo struct {
1595     Id    int64 `json:"id"`
1596     UserId int64 `json:"user_id"`
1597     Nickname string `json:"nickname"`
1598     Avatar string `json:"avatar"`
1599     RankLevel int `json:"rank_level"`
1600     RankName string `json:"rank_name"`

```

```

1601     CreatedAt string `json:"created_at"
1602 }
1603
1604 type FriendRequestInfo struct {
1605     Id      int64 `json:"id"
1606     FromUserId int64 `json:"from_user_id"
1607     Nickname string `json:"nickname"
1608     Avatar   string `json:"avatar"
1609     Message  string `json:"message"
1610     Status   int   `json:"status"
1611     CreatedAt string `json:"created_at"
1612 }
1613
1614 type GameTypeStats struct {
1615     GameType int   `json:"game_type"
1616     GameTypeName string `json:"game_type_name"
1617     TotalMatches int   `json:"total_matches"
1618     Wins      int   `json:"wins"
1619     Losses    int   `json:"losses"
1620     WinRate   float64 `json:"win_rate"
1621     HighestScore int   `json:"highest_score"
1622 }
1623
1624 type GetAchievementListReq struct {
1625     Category string `form:"category,optional"
1626 }
1627
1628 type GetAchievementListResp struct {
1629     Success bool   `json:"success"
1630     List   []AchievementDef `json:"list"
1631 }
1632
1633 type GetCurrentMatchResp struct {
1634     Success bool   `json:"success"
1635     Match  *CurrentMatchInfo `json:"match"
1636 }
1637
1638 type GetCurrentSeasonResp struct {
1639     Success bool   `json:"success"
1640     Season *SeasonInfo `json:"season"
1641 }
1642
1643 type GetEventNewsDetailReq struct {
1644     EventId int64 `form:"event_id,optional"
1645 }
1646
1647 type GetEventNewsDetailResp struct {
1648     Success bool   `json:"success"
1649     Event  *EventNewsInfo `json:"event"
1650     Stages []EventNewsStageInfo `json:"stages"

```

```

1651 }
1652
1653 type GetEventNewsListReq struct {
1654     Page    int    `form:"page,default=1"`
1655     PageSize int    `form:"page_size,default=20"`
1656     Status  int    `form:"status,optional,default=-1"`
1657     GameType int    `form:"game_type,optional"`
1658     City    string `form:"city,optional"`
1659 }
1660
1661 type GetEventNewsListResp struct {
1662     Success bool    `json:"success"`
1663     Total  int64    `json:"total"`
1664     List   []EventNewsInfo `json:"list"`
1665 }
1666
1667 type GetFeaturedEventNewsResp struct {
1668     Success bool    `json:"success"`
1669     Event   *EventNewsInfo `json:"event"`
1670 }
1671
1672 type GetFollowListReq struct {
1673     Page    int    `form:"page,default=1"`
1674     PageSize int    `form:"page_size,default=20"`
1675 }
1676
1677 type GetFollowListResp struct {
1678     Success bool    `json:"success"`
1679     Total  int64    `json:"total"`
1680     List   []FollowUserInfo `json:"list"`
1681 }
1682
1683 type GetFriendListReq struct {
1684     Page    int    `form:"page,default=1"`
1685     PageSize int    `form:"page_size,default=20"`
1686     Keyword string `form:"keyword,optional"`
1687 }
1688
1689 type GetFriendListResp struct {
1690     Success bool    `json:"success"`
1691     Total  int64    `json:"total"`
1692     List   []FriendInfo `json:"list"`
1693 }
1694
1695 type GetFriendRequestsReq struct {
1696     Page    int    `form:"page,default=1"`
1697     PageSize int    `form:"page_size,default=20"`
1698 }
1699
1700 type GetFriendRequestsResp struct {

```

```

1701     Success bool      `json:"success"
1702     Total  int64      `json:"total"
1703     List   []FriendRequestInfo `json:"list"
1704 }
1705
1706 type GetGlossaryReq struct {
1707     Category string `form:"category,optional"
1708 }
1709
1710 type GetGlossaryResp struct {
1711     Success bool      `json:"success"
1712     List   []GlossaryItemInfo `json:"list"
1713 }
1714
1715 type GetLeaderboardReq struct {
1716     GameType int `form:"game_type,optional,default=3"
1717     Page    int `form:"page,default=1"
1718     PageSize int `form:"page_size,default=20"
1719 }
1720
1721 type GetLeaderboardResp struct {
1722     Success bool      `json:"success"
1723     Total  int64      `json:"total"
1724     TopThree []LeaderboardItem `json:"top_three" // 前三名
1725     MyRanking *LeaderboardItem `json:"my_ranking" // 我的排名
1726     List   []LeaderboardItem `json:"list" // 第4名开始的列表
1727 }
1728
1729 type GetMatchDetailReq struct {
1730     MatchId int64 `form:"match_id"
1731 }
1732
1733 type GetMatchDetailResp struct {
1734     Success bool      `json:"success"
1735     Match  MatchDetailData `json:"match"
1736 }
1737
1738 type GetMatchDurationStatsReq struct {
1739     GameType int `form:"game_type,optional"
1740 }
1741
1742 type GetMatchDurationStatsResp struct {
1743     Success bool      `json:"success"
1744     Stats  *DurationStats `json:"stats"
1745 }
1746
1747 type GetMatchQRCodeResp struct {
1748     Success bool `json:"success"
1749     QrcodeData string `json:"qrcode_data" // 二维码内容JSON: {"user_id":xxx,"nickname":"xxx","avatar":"xxx","ts":xxx}
1750 }

```

```

1751
1752 type GetMatchShareDataReq struct {
1753     MatchId int64 `form:"match_id"`
1754 }
1755
1756 type GetMatchShareDataResp struct {
1757     Success bool    `json:"success"`
1758     Data    *MatchShareData `json:"data"`
1759 }
1760
1761 type GetMemberPlansResp struct {
1762     Success bool    `json:"success"`
1763     CurrentTime string  `json:"current_time"`
1764     Plans    []MemberPlanInfo `json:"plans"`
1765 }
1766
1767 type GetMemberStatusResp struct {
1768     Success bool `json:"success"`
1769     IsActive bool `json:"is_active"`
1770     CurrentTime string `json:"current_time"`
1771     PlanCode string `json:"plan_code,optional"`
1772     PlanName string `json:"plan_name,optional"`
1773     MemberExpiresAt string `json:"member_expires_at,optional"`
1774 }
1775
1776 type GetMemberSubscriptionOrderStatusReq struct {
1777     OrderNo string `form:"order_no"`
1778 }
1779
1780 type GetMemberSubscriptionOrderStatusResp struct {
1781     Success bool `json:"success"`
1782     Message string `json:"message,optional"`
1783     OrderNo string `json:"order_no"`
1784     PayChannel string `json:"pay_channel"`
1785     PlanCode string `json:"plan_code"`
1786     PlanName string `json:"plan_name"`
1787     AmountFen int `json:"amount_fen"`
1788     AmountYuan string `json:"amount_yuan"`
1789     Status string `json:"status"`
1790     PaidAt string `json:"paid_at,optional"`
1791     MemberExpiresAtBefore string `json:"member_expires_at_before,optional"`
1792     MemberExpiresAtAfter string `json:"member_expires_at_after,optional"`
1793 }
1794
1795 type GetMyCheckinsReq struct {
1796     Page int `form:"page,default=1"`
1797     PageSize int `form:"page_size,default=20"`
1798 }
1799
1800 type GetMyCheckinsResp struct {

```

```

1801     Success bool      `json:"success"
1802     Total  int64      `json:"total"
1803     List   []VenueCheckinInfo `json:"list"
1804 }
1805
1806 type GetMySeasonRecordReq struct {
1807     SeasonId int64 `form:"season_id,optional"
1808     GameType int  `form:"game_type,optional,default=3"
1809 }
1810
1811 type GetMySeasonRecordResp struct {
1812     Success bool      `json:"success"
1813     Record  *SeasonRecordInfo `json:"record"
1814 }
1815
1816 type GetMyTournamentsReq struct {
1817     Page   int `form:"page,default=1"
1818     PageSize int `form:"page_size,default=20"
1819     Status int `form:"status,optional"
1820 }
1821
1822 type GetNearbyVenuesReq struct {
1823     Latitude float64 `form:"latitude"
1824     Longitude float64 `form:"longitude"
1825     Radius   int  `form:"radius,optional,default=5000"
1826     Limit    int  `form:"limit,optional,default=10"
1827 }
1828
1829 type GetNotificationListReq struct {
1830     Page   int `form:"page,default=1"
1831     PageSize int `form:"page_size,default=20"
1832     Type   string `form:"type,optional"
1833 }
1834
1835 type GetNotificationListResp struct {
1836     Success bool      `json:"success"
1837     Total  int64      `json:"total"
1838     List   []NotificationInfo `json:"list"
1839 }
1840
1841 type GetOngoingMatchesReq struct {
1842     Page   int `form:"page,default=1"
1843     PageSize int `form:"page_size,default=20"
1844 }
1845
1846 type GetOngoingMatchesResp struct {
1847     Success bool      `json:"success"
1848     Total  int64      `json:"total"
1849     List   []OngoingMatchItem `json:"list"
1850 }

```

```

1851
1852 type GetOpponentListReq struct {
1853     Page    int    `form:"page,default=1"`
1854     PageSize int    `form:"page_size,default=20"`
1855     Keyword  string `form:"keyword,optional"`
1856     TargetUserId int64 `form:"target_user_id,optional"`
1857 }
1858
1859 type GetOpponentListResp struct {
1860     Success bool    `json:"success"`
1861     Message string   `json:"message,optional"`
1862     Hidden  bool    `json:"hidden"`
1863     TotalOpponents int    `json:"total_opponents" // 总对手数`
1864     TotalWins  int    `json:"total_wins" // 总胜场`
1865     Total     int64   `json:"total"`
1866     List      []OpponentRecordItem `json:"list"`
1867 }
1868
1869 type GetOpponentStrengthResp struct {
1870     Success bool    `json:"success"`
1871     List    []OpponentStrengthItem `json:"list"`
1872 }
1873
1874 type GetPendingChallengesResp struct {
1875     Success bool    `json:"success"`
1876     List    []ChallengeInfo `json:"list"`
1877 }
1878
1879 type GetPostCommentsReq struct {
1880     PostId int64 `form:"post_id"`
1881     Page   int   `form:"page,default=1"`
1882     PageSize int   `form:"page_size,default=20"`
1883 }
1884
1885 type GetPostCommentsResp struct {
1886     Success bool    `json:"success"`
1887     Total  int64   `json:"total"`
1888     List   []PostCommentInfo `json:"list"`
1889 }
1890
1891 type GetPostListReq struct {
1892     Page    int    `form:"page,default=1"`
1893     PageSize int    `form:"page_size,default=20"`
1894 }
1895
1896 type GetPostListResp struct {
1897     Success bool    `json:"success"`
1898     Total  int64   `json:"total"`
1899     List   []SocialPostInfo `json:"list"`
1900 }

```

```

1901
1902 type GetPublicMatchDetailReq struct {
1903     MatchId int64 `form:"match_id"`
1904 }
1905
1906 type GetPublicMatchDetailResp struct {
1907     Success bool          `json:"success"`
1908     Match   *PublicMatchDetailData `json:"match"`
1909 }
1910
1911 type GetRankListReq struct {
1912     GameType int `form:"game_type,optional,default=3"`
1913 }
1914
1915 type GetRankListResp struct {
1916     Success bool    `json:"success"`
1917     List   []RankItem `json:"list"`
1918 }
1919
1920 type GetRankScoreTrendReq struct {
1921     Limit   int `form:"limit,optional,default=30"`
1922     GameType int `form:"game_type,optional,default=3"`
1923 }
1924
1925 type GetRankScoreTrendResp struct {
1926     Success bool    `json:"success"`
1927     List   []TrendPoint `json:"list"`
1928 }
1929
1930 type GetRecentTrendReq struct {
1931     Limit   int `form:"limit,optional,default=30"`
1932     GameType int `form:"game_type,optional"`
1933 }
1934
1935 type GetRecentTrendResp struct {
1936     Success bool    `json:"success"`
1937     List   []TrendPoint `json:"list"`
1938 }
1939
1940 type GetRuleCategoriesResp struct {
1941     Success bool    `json:"success"`
1942     List   []RuleCategoryInfo `json:"list"`
1943 }
1944
1945 type GetRuleContentReq struct {
1946     Category string `form:"category"`
1947     ContentType string `form:"content_type"`
1948 }
1949
1950 type GetRuleContentResp struct {

```

```

1951     Success bool      `json:"success"
1952     List  []RuleContentItem `json:"list"
1953 }
1954
1955 type GetSeasonLeaderboardReq struct {
1956     SeasonId int64 `form:"season_id,optional"
1957     GameType int  `form:"game_type,optional,default=3"
1958     Page    int  `form:"page,default=1"
1959     PageSize int  `form:"page_size,default=20"
1960 }
1961
1962 type GetSeasonLeaderboardResp struct {
1963     Success bool      `json:"success"
1964     Season *SeasonInfo `json:"season"
1965     Total  int64      `json:"total"
1966     List  []SeasonLeaderboardItem `json:"list"
1967 }
1968
1969 type GetSeasonReportReq struct {
1970     SeasonId int64 `form:"season_id"
1971     GameType int  `form:"game_type,optional,default=3"
1972 }
1973
1974 type GetSeasonReportResp struct {
1975     Success bool      `json:"success"
1976     Report *SeasonReportData `json:"report"
1977 }
1978
1979 type GetSingleHighScoreReq struct {
1980     GameType int `form:"game_type,optional"
1981     Limit    int `form:"limit,optional,default=10"
1982 }
1983
1984 type GetSingleHighScoreResp struct {
1985     Success bool      `json:"success"
1986     List  []SingleHighScoreRecord `json:"list"
1987 }
1988
1989 type GetStatsByGameTypeResp struct {
1990     Success bool      `json:"success"
1991     List  []GameTypeStats `json:"list"
1992 }
1993
1994 type GetTournamentBracketReq struct {
1995     TournamentId int64 `form:"tournament_id"
1996 }
1997
1998 type GetTournamentBracketResp struct {
1999     Success bool      `json:"success"
2000     Matches []TournamentBracketMatch `json:"matches"

```

```

2001     TotalRounds int           `json:"total_rounds"`
2002 }
2003
2004 type GetTournamentDetailReq struct {
2005     TournamentId int64 `form:"tournament_id"`
2006 }
2007
2008 type GetTournamentDetailResp struct {
2009     Success bool           `json:"success"`
2010     Tournament *TournamentInfo `json:"tournament"`
2011     Participants []TournamentParticipant `json:"participants"`
2012 }
2013
2014 type GetTournamentListReq struct {
2015     Page int `form:"page,default=1"`
2016     PageSize int `form:"page_size,default=20"`
2017     Status int `form:"status,optional"`
2018     GameType int `form:"game_type,optional"`
2019     City string `form:"city,optional"`
2020 }
2021
2022 type GetTournamentListResp struct {
2023     Success bool           `json:"success"`
2024     Total int64           `json:"total"`
2025     List []TournamentInfo `json:"list"`
2026 }
2027
2028 type GetTournamentShareDataReq struct {
2029     TournamentId int64 `form:"tournament_id"`
2030 }
2031
2032 type GetTournamentShareDataResp struct {
2033     Success bool           `json:"success"`
2034     Data *TournamentShareData `json:"data"`
2035 }
2036
2037 type GetUnreadCountResp struct {
2038     Success bool `json:"success"`
2039     Count int `json:"count"`
2040 }
2041
2042 type GetUserAchievementsResp struct {
2043     Success bool           `json:"success"`
2044     Total int             `json:"total"`
2045     List []AchievementDef `json:"list"`
2046 }
2047
2048 type GetUserInfoResp struct {
2049     Success bool           `json:"success"`
2050     UserInfo *UserInfo `json:"user_info"`

```

```

2051  }
2052
2053  type GetUserPrivacyResp struct {
2054      Success    bool `json:"success"`
2055      HideMatchRecord bool `json:"hide_match_record"`
2056  }
2057
2058  type GetUserRankInfoReq struct {
2059      GameType int `form:"game_type,optional,default=3"`
2060  }
2061
2062  type GetUserRankInfoResp struct {
2063      Success bool `json:"success"`
2064      RankInfo *RankInfo `json:"rank_info"`
2065  }
2066
2067  type GetUserTitlesResp struct {
2068      Success bool `json:"success"`
2069      List []TitleInfo `json:"list"`
2070  }
2071
2072  type GetVenueDetailReq struct {
2073      VenueId int64 `form:"venue_id"`
2074  }
2075
2076  type GetVenueDetailResp struct {
2077      Success bool `json:"success"`
2078      Venue *VenueInfo `json:"venue"`
2079  }
2080
2081  type GetVenueListReq struct {
2082      Page int `form:"page,default=1"`
2083      PageSize int `form:"page_size,default=20"`
2084      City string `form:"city,optional"`
2085      Latitude float64 `form:"latitude,optional"`
2086      Longitude float64 `form:"longitude,optional"`
2087      SortBy string `form:"sort_by,optional,default=distance"`
2088  }
2089
2090  type GetVenueListResp struct {
2091      Success bool `json:"success"`
2092      Total int64 `json:"total"`
2093      List []VenueInfo `json:"list"`
2094  }
2095
2096  type GlossaryItemInfo struct {
2097      Id int64 `json:"id"`
2098      Title string `json:"title"`
2099      Content string `json:"content"`
2100      Category string `json:"category"`

```

```

2101  }
2102
2103  type H2HHistoryReq struct {
2104      TargetUserId int64 `form:"target_user_id,optional"`
2105      OpponentId   int64 `form:"opponent_id,optional"`
2106      OpponentName string `form:"opponent_name,optional"`
2107      Page        int    `form:"page,default=1"`
2108      PageSize    int    `form:"page_size,default=20"`
2109      Result      int    `form:"result,optional"`
2110  }
2111
2112  type H2HHistoryResp struct {
2113      Success bool    `json:"success"`
2114      Message string   `json:"message,optional"`
2115      Total   int64    `json:"total"`
2116      List   []MatchListItem `json:"list"`
2117  }
2118
2119  type H2HOpponent struct {
2120      Id   int64 `json:"id"`
2121      Name string `json:"name"`
2122      Avatar string `json:"avatar"`
2123  }
2124
2125  type H2HStats struct {
2126      TotalMatches int `json:"total_matches"`
2127      MyWins      int `json:"my_wins"`
2128      OpponentWins int `json:"opponent_wins"`
2129      WinRate     float64 `json:"win_rate"`
2130      AvgScoreDiff float64 `json:"avg_score_diff"`
2131      MaxWinStreak int `json:"max_win_streak"`
2132  }
2133
2134  type H2HStatsReq struct {
2135      TargetUserId int64 `form:"target_user_id,optional"`
2136      OpponentId   int64 `form:"opponent_id,optional"`
2137      OpponentName string `form:"opponent_name,optional"`
2138  }
2139
2140  type H2HStatsResp struct {
2141      Success bool    `json:"success"`
2142      Message string   `json:"message,optional"`
2143      Opponent *H2HOpponent `json:"opponent"`
2144      Stats    *H2HStats    `json:"stats"`
2145  }
2146
2147  type HandleChallengeReq struct {
2148      ChallengeId int64 `json:"challenge_id"`
2149  }
2150

```

```

2151 type HandleFriendRequestReq struct {
2152     RequestId int64 `json:"request_id"`
2153 }
2154
2155 type LeaderboardItem struct {
2156     Rank    int    `json:"rank" // 排名
2157     UserId  int64  `json:"user_id"
2158     Nickname string `json:"nickname"
2159     Avatar  string `json:"avatar"
2160     RankLevel int  `json:"rank_level" // 段位等级
2161     RankName string `json:"rank_name" // 段位名称
2162     RankScore int  `json:"rank_score" // 排位分
2163     WinRate  int  `json:"win_rate" // 胜率百分比
2164 }
2165
2166 type LoginByOauthReq struct {
2167     Provider string `json:"provider" // huawei
2168     OpenId   string `json:"open_id"
2169     UnionId   string `json:"union_id,optional"
2170     NickName  string `json:"nick_name,optional"
2171     AvatarUrl string `json:"avatar_url,optional"
2172     Platform  string `json:"platform,optional"
2173 }
2174
2175 type LoginByOauthResp struct {
2176     Success    bool    `json:"success"
2177     AccessToken string  `json:"access_token"
2178     RefreshToken string `json:"refresh_token"
2179     ExpiresIn  int64   `json:"expires_in"
2180     NeedBindPhone bool    `json:"need_bind_phone"
2181     UserInfo   *UserInfo `json:"user_info"
2182 }
2183
2184 type LoginReq struct {
2185     Phone string `json:"phone"
2186     SmsCode string `json:"sms_code"
2187 }
2188
2189 type LoginResp struct {
2190     Success    bool    `json:"success"
2191     AccessToken string  `json:"access_token"
2192     RefreshToken string `json:"refresh_token"
2193     ExpiresIn  int64   `json:"expires_in"
2194     UserInfo   *UserInfo `json:"user_info"
2195 }
2196
2197 type MatchAchievement struct {
2198     BreakClear int `json:"break_clear" // 炸清次数
2199     ContinueClear int `json:"continue_clear" // 接清次数
2200     GoldenBreak int `json:"golden_break" // 金球数

```

```

2201     NineOnBreak  int `json:"nine_on_break" // 大金数
2202     Break50      int `json:"break_50"    // 单杆50+
2203     Break100     int `json:"break_100"   // 单杆100+
2204     Break147     int `json:"break_147"   // 单杆147
2205 }
2206
2207 type MatchDetailData struct {
2208     Id             int64    `json:"id"
2209     GameType       int      `json:"game_type"
2210     Status         int      `json:"status" // 1=进行中 2=已完成 3=已取消
2211     ServerRevision int64    `json:"server_revision"
2212     IsPlayer1      bool    `json:"is_player1" // 当前用户是否是创建者(用于视角判断)
2213     MyScore        int      `json:"my_score"   // 从当前用户视角的分数
2214     OpponentScore  int      `json:"opponent_score" // 从当前用户视角的对手分数
2215     MyName         string   `json:"my_name"
2216     OpponentName   string   `json:"opponent_name"
2217     MyAvatar       string   `json:"my_avatar"
2218     OpponentAvatar string   `json:"opponent_avatar"
2219     MyWinRate      float64  `json:"my_win_rate" // 我方胜率(百分比)
2220     OpponentWinRate float64  `json:"opponent_win_rate" // 对手胜率(百分比)
2221     MyMaxScore     int      `json:"my_max_score" // 我方单杆最高
2222     OpponentMaxScore int    `json:"opponent_max_score" // 对手单杆最高
2223     MyRankChange   int      `json:"my_rank_change"
2224     OpponentRankChange int    `json:"opponent_rank_change"
2225     MyRankDetails  []RankDetail `json:"my_rank_details"
2226     CurrentFrameStarted bool    `json:"current_frame_started,optional"
2227     CurrentFrameMyScore int    `json:"current_frame_my_score,optional"
2228     CurrentFrameOpponentScore int    `json:"current_frame_opponent_score,optional"
2229     RedBallCount   int      `json:"red_ball_count,optional"
2230     SnookerClearanceStarted bool    `json:"snooker_clearance_started,optional"
2231     SnookerClearedColors []int    `json:"snooker_cleared_colors,optional"
2232     SnookerExpectedClearanceScore int    `json:"snooker_expected_clearance_score,optional"
2233     SnookerClearanceCompleted bool    `json:"snooker_clearance_completed,optional"
2234     SummaryHighlights []MatchSummaryItem `json:"summary_highlights,optional"
2235     SummaryStats     []MatchSummaryItem `json:"summary_stats,optional"
2236     Achievements     MatchAchievement `json:"achievements"
2237     CreatedAt        string    `json:"created_at"
2238 }
2239
2240 type MatchFoulReq struct {
2241     MatchId  int64 `json:"match_id"
2242     Actor    int   `json:"actor" // 犯规方
2243     Score    int   `json:"score,optional" // 犯规加分, 默认1
2244     ClientActionId string `json:"client_action_id"
2245     BaseRevision int64 `json:"base_revision"
2246 }
2247
2248 type MatchListItem struct {
2249     Id      int64 `json:"id"
2250     OpponentId int64 `json:"opponent_id" // 对手ID (注册用户才有)

```

```

2251   GameType   int   `json:"game_type"
2252   GameTypeName string `json:"game_type_name"
2253   OpponentName string `json:"opponent_name"
2254   OpponentAvatar string `json:"opponent_avatar"
2255   MyScore     int   `json:"my_score"
2256   OpponentScore int `json:"opponent_score"
2257   Result      int   `json:"result"
2258   MatchTime   string `json:"match_time"
2259 }
2260
2261 type MatchListReq struct {
2262     Page   int `form:"page,default=1"
2263     PageSize int `form:"page_size,default=20"
2264     GameType int `form:"game_type,optional"
2265     Result int `form:"result,optional"
2266 }
2267
2268 type MatchListResp struct {
2269     Success bool `json:"success"
2270     Total int64 `json:"total"
2271     List []MatchListItem `json:"list"
2272 }
2273
2274 type MatchScoreReq struct {
2275     MatchId int64 `json:"match_id"
2276     Actor int `json:"actor" // 1=我 2=对手
2277     Score int `json:"score" // 加的分
2278     ClientActionId string `json:"client_action_id"
2279     BaseRevision int64 `json:"base_revision"
2280 }
2281
2282 type MatchScoreResp struct {
2283     Accepted bool `json:"accepted"
2284     Success bool `json:"success"
2285     Message string `json:"message,optional"
2286     ClientActionId string `json:"client_action_id,optional"
2287     ServerRevision int64 `json:"server_revision"
2288     Snapshot MatchSyncSnapshot `json:"snapshot"
2289     RedBallCount int `json:"red_ball_count,optional"
2290     SnookerClearanceStarted bool `json:"snooker_clearance_started,optional"
2291     SnookerClearedColors []int `json:"snooker_cleared_colors,optional"
2292     SnookerExpectedClearanceScore int `json:"snooker_expected_clearance_score,optional"
2293     SnookerClearanceCompleted bool `json:"snooker_clearance_completed,optional"
2294     CurrentFrameStarted bool `json:"current_frame_started,optional"
2295     CurrentFrameMyScore int `json:"current_frame_my_score,optional"
2296     CurrentFrameOpponentScore int `json:"current_frame_opponent_score,optional"
2297     MyScore int `json:"my_score"
2298     OpponentScore int `json:"opponent_score"
2299 }
2300

```

```

2301 type MatchShareData struct {
2302     MatchId      int64      `json:"match_id"`
2303     GameType      int        `json:"game_type"`
2304     GameTypeName  string     `json:"game_type_name"`
2305     MyName        string     `json:"my_name"`
2306     MyAvatar      string     `json:"my_avatar"`
2307     MyScore       int        `json:"my_score"`
2308     OpponentName  string     `json:"opponent_name"`
2309     OpponentAvatar string     `json:"opponent_avatar"`
2310     OpponentScore int        `json:"opponent_score"`
2311     Result        string     `json:"result"`
2312     MatchTime     string     `json:"match_time"`
2313     RankChange    int        `json:"rank_change"`
2314     SummaryHighlights []MatchSummaryItem `json:"summary_highlights,optional"`
2315     SummaryStats  []MatchSummaryItem `json:"summary_stats,optional"`
2316 }
2317
2318 type MatchSummaryItem struct {
2319     Label string `json:"label"`
2320     Value string `json:"value"`
2321     Desc string `json:"desc,optional"`
2322     Icon string `json:"icon,optional"`
2323     Tone string `json:"tone,optional"`
2324 }
2325
2326 type MatchSyncSnapshot struct {
2327     MatchId      int64 `json:"match_id"`
2328     Status       int   `json:"status"`
2329     ServerRevision int64 `json:"server_revision"`
2330     MyScore      int   `json:"my_score"`
2331     OpponentScore int   `json:"opponent_score"`
2332     CurrentFrameStarted bool `json:"current_frame_started,optional"`
2333     CurrentFrameMyScore int   `json:"current_frame_my_score,optional"`
2334     CurrentFrameOpponentScore int `json:"current_frame_opponent_score,optional"`
2335     CurrentRound    int   `json:"current_round"`
2336     TotalRounds     int   `json:"total_rounds"`
2337     RedBallCount    int   `json:"red_ball_count,optional"`
2338     SnookerClearanceStarted bool `json:"snooker_clearance_started,optional"`
2339     SnookerClearedColors []int `json:"snooker_cleared_colors,optional"`
2340     SnookerExpectedClearanceScore int `json:"snooker_expected_clearance_score,optional"`
2341     SnookerClearanceCompleted bool `json:"snooker_clearance_completed,optional"`
2342 }
2343
2344 type MatchUndoReq struct {
2345     MatchId      int64 `json:"match_id"`
2346     ClientActionId string `json:"client_action_id"`
2347     BaseRevision int64 `json:"base_revision"`
2348 }
2349
2350 type MatchUndoResp struct {

```

```

2351     Accepted      bool      `json:"accepted"
2352     Success       bool      `json:"success"
2353     ClientActionId string    `json:"client_action_id,optional"
2354     ServerRevision int64     `json:"server_revision"
2355     Snapshot       MatchSyncSnapshot `json:"snapshot"
2356     CurrentFrameStarted bool    `json:"current_frame_started,optional"
2357     CurrentFrameMyScore int     `json:"current_frame_my_score,optional"
2358     CurrentFrameOpponentScore int    `json:"current_frame_opponent_score,optional"
2359     MyScore        int     `json:"my_score"
2360     OpponentScore  int     `json:"opponent_score"
2361     Message       string   `json:"message"
2362 }
2363
2364 type MemberPlanInfo struct {
2365     PlanCode string `json:"plan_code"
2366     PlanName string `json:"plan_name"
2367     PriceFen int    `json:"price_fen"
2368     PriceYuan string `json:"price_yuan"
2369     DurationDays int   `json:"duration_days"
2370     DurationLabel string `json:"duration_label"
2371     Description string `json:"description"
2372     Highlight string `json:"highlight,optional"
2373 }
2374
2375 type NotificationIdReq struct {
2376     NotificationId int64 `json:"notification_id"
2377 }
2378
2379 type NotificationInfo struct {
2380     Id int64 `json:"id"
2381     Type string `json:"type"
2382     Title string `json:"title"
2383     Content string `json:"content"
2384     Data string `json:"data"
2385     IsRead bool `json:"is_read"
2386     CreatedAt string `json:"created_at"
2387 }
2388
2389 type OngoingMatchItem struct {
2390     Id int64 `json:"id"
2391     GameType int `json:"game_type"
2392     GameTypeName string `json:"game_type_name"
2393     Player1Id int64 `json:"player1_id" // 玩家1(创建者)ID
2394     Player1Name string `json:"player1_name"
2395     Player1Avatar string `json:"player1_avatar"
2396     Player2Id int64 `json:"player2_id" // 玩家2(对手)ID, 0表示非注册用户
2397     Player2Name string `json:"player2_name"
2398     Player2Avatar string `json:"player2_avatar"
2399     Player1Score int `json:"player1_score"
2400     Player2Score int `json:"player2_score"

```

```

2401     CurrentRound int `json:"current_round"
2402     MatchTime string `json:"match_time" // 开始时间 ISO格式
2403     DurationSeconds int64 `json:"duration_seconds" // 对局已持续秒数，由服务端计算
2404 }
2405
2406 type OpponentItem struct {
2407     Id int64 `json:"id"
2408     Name string `json:"name"
2409     Avatar string `json:"avatar"
2410     MatchCount int `json:"match_count"
2411 }
2412
2413 type OpponentRecordItem struct {
2414     Id int64 `json:"id"
2415     Name string `json:"name"
2416     Avatar string `json:"avatar"
2417     TotalMatches int `json:"total_matches" // 总对局数
2418     Wins int `json:"wins" // 我方胜场
2419     Losses int `json:"losses" // 我方负场
2420     WinRate float64 `json:"win_rate" // 胜率百分比
2421     LastMatchAt string `json:"last_match_at" // 上次对局时间
2422 }
2423
2424 type OpponentStrengthItem struct {
2425     RankRange string `json:"rank_range"
2426     Matches int `json:"matches"
2427     Wins int `json:"wins"
2428     WinRate float64 `json:"win_rate"
2429 }
2430
2431 type PostCommentInfo struct {
2432     Id int64 `json:"id"
2433     UserId int64 `json:"user_id"
2434     Nickname string `json:"nickname"
2435     Avatar string `json:"avatar"
2436     Content string `json:"content"
2437     CreatedAt string `json:"created_at"
2438 }
2439
2440 type PostIdReq struct {
2441     PostId int64 `json:"post_id"
2442 }
2443
2444 type PublicMatchDetailData struct {
2445     Id int64 `json:"id"
2446     GameType int `json:"game_type"
2447     Status int `json:"status" // 1=进行中 2=已完成 3=已取消
2448     ServerRevision int64 `json:"server_revision"
2449     Player1Id int64 `json:"player1_id"
2450     Player1Name string `json:"player1_name"

```

```

2451   Player1Avatar    string   `json:"player1_avatar"
2452   Player2Id        int64    `json:"player2_id"
2453   Player2Name       string   `json:"player2_name"
2454   Player2Avatar      string   `json:"player2_avatar"
2455   Player1Score       int      `json:"player1_score"
2456   Player2Score       int      `json:"player2_score"
2457   CurrentFramePlayer1Score int    `json:"current_frame_player1_score,optional"
2458   CurrentFramePlayer2Score int    `json:"current_frame_player2_score,optional"
2459   CurrentFrameStarted bool    `json:"current_frame_started,optional"
2460   CurrentRound       int      `json:"current_round"
2461   TotalRounds        int      `json:"total_rounds"
2462   DurationSeconds     int64    `json:"duration_seconds"
2463   Rounds              []RoundRecord `json:"rounds" // 局记录
2464   CreatedAt          string   `json:"created_at"
2465 }
2466
2467 type RankDetail struct {
2468     Label string `json:"label"
2469     Value int   `json:"value"
2470 }
2471
2472 type RankInfo struct {
2473     Level int   `json:"level" // 段位等级 1-5
2474     Name  string `json:"name"  // 段位名称
2475     Icon  string `json:"icon"  // 段位图标URL
2476     RankScore int `json:"rank_score" // 当前排位分
2477     TotalWins int `json:"total_wins" // 总胜场
2478     TotalLosses int `json:"total_losses" // 总败场
2479     MaxStreak int `json:"max_streak" // 最高连胜
2480     NextLevel int `json:"next_level" // 下一段位等级
2481     NextName string `json:"next_name" // 下一段位名称
2482     NextScore int `json:"next_score" // 晋级所需分数
2483     Progress int `json:"progress" // 晋级进度百分比
2484 }
2485
2486 type RankItem struct {
2487     Level int   `json:"level"
2488     Name  string `json:"name"
2489     Icon  string `json:"icon"
2490     MinScore int `json:"min_score" // 晋升最低分
2491     IsCurrent bool `json:"is_current" // 是否当前段位
2492 }
2493
2494 type RoundRecord struct {
2495     RoundNumber int `json:"round_number"
2496     Player1Score int `json:"player1_score"
2497     Player2Score int `json:"player2_score"
2498     Winner int `json:"winner" // 1=玩家1胜 2=玩家2胜
2499 }
2500

```

```

2501 type RuleCategoryInfo struct {
2502     Category string `json:"category"`
2503     Name      string `json:"name"`
2504     Description string `json:"description"`
2505     RuleCount int   `json:"rule_count"`
2506     FoulCount int   `json:"foul_count"`
2507     GlossaryCount int `json:"glossary_count"`
2508 }
2509
2510 type RuleContentItem struct {
2511     Id      int64 `json:"id"`
2512     Title   string `json:"title"`
2513     Content string `json:"content"`
2514     SortOrder int `json:"sort_order"`
2515 }
2516
2517 type SearchOpponentReq struct {
2518     Keyword string `form:"keyword"`
2519 }
2520
2521 type SearchOpponentResp struct {
2522     Success bool   `json:"success"`
2523     List []OpponentItem `json:"list"`
2524 }
2525
2526 type SearchRulesReq struct {
2527     Keyword string `form:"keyword"`
2528 }
2529
2530 type SearchRulesResp struct {
2531     Success bool   `json:"success"`
2532     List []RuleContentItem `json:"list"`
2533 }
2534
2535 type SearchUserItem struct {
2536     UserId int64 `json:"user_id"`
2537     Nickname string `json:"nickname"`
2538     Avatar string `json:"avatar"`
2539     RankName string `json:"rank_name"`
2540     IsFriend bool `json:"is_friend"`
2541 }
2542
2543 type SearchUserReq struct {
2544     Keyword string `form:"keyword"`
2545 }
2546
2547 type SearchUserResp struct {
2548     Success bool   `json:"success"`
2549     List []SearchUserItem `json:"list"`
2550 }

```

```

2551
2552 type SeasonInfo struct {
2553     Id      int64 `json:"id"`
2554     Name     string `json:"name"`
2555     StartDate string `json:"start_date"`
2556     EndDate  string `json:"end_date"`
2557     Status   int    `json:"status"`
2558     RankResetRatio float64 `json:"rank_reset_ratio"`
2559 }
2560
2561 type SeasonLeaderboardItem struct {
2562     Rank    int    `json:"rank"`
2563     UserId  int64  `json:"user_id"`
2564     Nickname string `json:"nickname"`
2565     Avatar  string `json:"avatar"`
2566     RankScore int   `json:"rank_score"`
2567     PeakScore int   `json:"peak_score"`
2568     MatchesPlayed int  `json:"matches_played"`
2569     Wins    int   `json:"wins"`
2570 }
2571
2572 type SeasonRecordInfo struct {
2573     SeasonId int64 `json:"season_id"`
2574     SeasonName string `json:"season_name"`
2575     StartRankScore int `json:"start_rank_score"`
2576     EndRankScore int `json:"end_rank_score"`
2577     PeakRankScore int `json:"peak_rank_score"`
2578     MatchesPlayed int `json:"matches_played"`
2579     Wins    int `json:"wins"`
2580     FinalRank int `json:"final_rank"`
2581 }
2582
2583 type SeasonReportData struct {
2584     Record *SeasonRecordInfo `json:"record"`
2585     RankTrend []int `json:"rank_trend"`
2586     WinRateByType map[string]int `json:"win_rate_by_type,omitempty"`
2587     TopAchievements []AchievementDef `json:"top_achievements"`
2588 }
2589
2590 type SendChallengeReq struct {
2591     ToUserId int64 `json:"to_user_id"`
2592     GameType int   `json:"game_type"`
2593     Message string `json:"message,optional"`
2594 }
2595
2596 type SendChallengeResp struct {
2597     Success bool `json:"success"`
2598     ChallengeId int64 `json:"challenge_id"`
2599 }
2600

```

```

2601 type SendFriendRequestReq struct {
2602     ToUserId int64 `json:"to_user_id"`
2603     Message string `json:"message,optional"`
2604 }
2605
2606 type SendSmsReq struct {
2607     Phone string `json:"phone"`
2608     Scene string `json:"scene,optional,default=login" // login/bind`
2609 }
2610
2611 type SendSmsResp struct {
2612     Success bool `json:"success"`
2613     Message string `json:"message"`
2614 }
2615
2616 type SingleHighScoreRecord struct {
2617     MatchId int64 `json:"match_id"`
2618     Score int `json:"score"`
2619     GameType int `json:"game_type"`
2620     GameTypeName string `json:"game_type_name"`
2621     OpponentName string `json:"opponent_name"`
2622     Date string `json:"date"`
2623 }
2624
2625 type SocialPostInfo struct {
2626     Id int64 `json:"id"`
2627     UserId int64 `json:"user_id"`
2628     Nickname string `json:"nickname"`
2629     Avatar string `json:"avatar"`
2630     Content string `json:"content"`
2631     Images []string `json:"images"`
2632     PostType int `json:"post_type"`
2633     MatchId int64 `json:"match_id,omitempty"`
2634     LikesCount int `json:"likes_count"`
2635     CommentsCount int `json:"comments_count"`
2636     IsLiked bool `json:"is_liked"`
2637     Status int `json:"status"`
2638     StatusText string `json:"status_text"`
2639     RejectReason string `json:"reject_reason,optional"`
2640     IsMine bool `json:"is_mine"`
2641     CreatedAt string `json:"created_at"`
2642 }
2643
2644 type StartMatchReq struct {
2645     GameType int `json:"game_type" // 1=斯诺克 2=九球追分 3=中式八球 4=美式九球`
2646     GameMode string `json:"game_mode,optional" // 比赛模式`
2647     OpponentId int64 `json:"opponent_id,optional" // 对手ID`
2648     OpponentName string `json:"opponent_name" // 对手昵称`
2649     OpponentAvatar string `json:"opponent_avatar,optional" // 对手头像`
2650 }

```

```

2651
2652 type StartMatchResp struct {
2653     Success    bool        `json:"success"`
2654     Action     string      `json:"action,optional" // created/resume_existing/blocked`
2655     BlockReason string      `json:"block_reason,optional" // self_ongoing/opponent_ongoing`
2656     Message    string      `json:"message,optional"`
2657     MatchId    int64       `json:"match_id"`
2658     OngoingMatch *CurrentMatchInfo `json:"ongoing_match,optional"`
2659 }
2660
2661 type StartNextRoundReq struct {
2662     MatchId    int64 `json:"match_id"`
2663     ClientActionId string `json:"client_action_id"`
2664     BaseRevision int64 `json:"base_revision"`
2665 }
2666
2667 type StartNextRoundResp struct {
2668     Accepted    bool        `json:"accepted"`
2669     Success     bool        `json:"success"`
2670     ClientActionId string      `json:"client_action_id,optional"`
2671     ServerRevision int64       `json:"server_revision"`
2672     Snapshot     MatchSyncSnapshot `json:"snapshot"`
2673     RoundNo      int          `json:"round_no" // 新局数`
2674     CurrentFrameStarted bool        `json:"current_frame_started,optional"`
2675     CurrentFrameMyScore int         `json:"current_frame_my_score,optional"`
2676     CurrentFrameOpponentScore int        `json:"current_frame_opponent_score,optional"`
2677     MyScore      int         `json:"my_score" // 我方总分`
2678     OpponentScore int         `json:"opponent_score" // 对手总分`
2679 }
2680
2681 type TitleInfo struct {
2682     Id    int64 `json:"id"`
2683     TitleName string `json:"title_name"`
2684     Source string `json:"source"`
2685     Equipped bool `json:"equipped"`
2686     CreatedAt string `json:"created_at"`
2687 }
2688
2689 type TournamentBracketMatch struct {
2690     Id    int64 `json:"id"`
2691     RoundNumber int `json:"round_number"`
2692     MatchOrder int `json:"match_order"`
2693     Player1Id int64 `json:"player1_id"`
2694     Player1Name string `json:"player1_name"`
2695     Player2Id int64 `json:"player2_id"`
2696     Player2Name string `json:"player2_name"`
2697     WinnerId int64 `json:"winner_id"`
2698     BracketPosition string `json:"bracket_position"`
2699     Status int `json:"status"`
2700 }

```

```

2701
2702 type TournamentIdReq struct {
2703     TournamentId int64 `json:"tournament_id"`
2704 }
2705
2706 type TournamentInfo struct {
2707     Id      int64 `json:"id"`
2708     CreatorId  int64 `json:"creator_id"`
2709     CreatorName string `json:"creator_name"`
2710     Name      string `json:"name"`
2711     Description string `json:"description"`
2712     GameType  int   `json:"game_type"`
2713     Format    int   `json:"format"`
2714     MaxPlayers int   `json:"max_players"`
2715     CurrentPlayers int `json:"current_players"`
2716     Status    int   `json:"status"`
2717     City      string `json:"city"`
2718     VenueName string `json:"venue_name"`
2719     StartTime string `json:"start_time"`
2720     EndTime   string `json:"end_time"`
2721     CreatedAt string `json:"created_at"`
2722 }
2723
2724 type TournamentParticipant struct {
2725     UserId int64 `json:"user_id"`
2726     Nickname string `json:"nickname"`
2727     Avatar string `json:"avatar"`
2728     Seed int `json:"seed"`
2729     Status int `json:"status"`
2730     FinalRank int `json:"final_rank"`
2731 }
2732
2733 type TournamentShareData struct {
2734     TournamentId int64 `json:"tournament_id"`
2735     TournamentName string `json:"tournament_name"`
2736     FinalRank int `json:"final_rank"`
2737     TotalPlayers int `json:"total_players"`
2738     MatchesPlayed int `json:"matches_played"`
2739     MatchesWon int `json:"matches_won"`
2740 }
2741
2742 type TrendPoint struct {
2743     MatchId int64 `json:"match_id"`
2744     Date string `json:"date"`
2745     WinRate float64 `json:"win_rate"`
2746     RankScore int `json:"rank_score,omitempty"`
2747     Result int `json:"result"`
2748 }
2749
2750 type UpdateNicknameReq struct {

```

```

2751     Nickname string `json:"nickname"
2752 }
2753
2754 type UpdateNicknameResp struct {
2755     Success bool `json:"success"
2756     Message string `json:"message"
2757     UserInfo *UserInfo `json:"user_info"
2758 }
2759
2760 type UpdatePushTokenReq struct {
2761     PushClientId string `json:"push_client_id"
2762 }
2763
2764 type UpdateTournamentMatchReq struct {
2765     TournamentMatchId int64 `json:"tournament_match_id"
2766     WinnerId int64 `json:"winner_id"
2767     MatchId int64 `json:"match_id,optional"
2768 }
2769
2770 type UpdateUserPrivacyReq struct {
2771     HideMatchRecord bool `json:"hide_match_record"
2772 }
2773
2774 type UpdateUserPrivacyResp struct {
2775     Success bool `json:"success"
2776     Message string `json:"message"
2777     HideMatchRecord bool `json:"hide_match_record"
2778 }
2779
2780 type UserInfo struct {
2781     Id int64 `json:"id"
2782     Phone string `json:"phone"
2783     Nickname string `json:"nickname"
2784     Avatar string `json:"avatar"
2785     Status int `json:"status"
2786     CreatedAt string `json:"created_at"
2787 }
2788
2789 type UserStatsResp struct {
2790     Success bool `json:"success"
2791     TotalMatches int `json:"total_matches" // 总对局数
2792     Wins int `json:"wins" // 胜场数
2793     Losses int `json:"losses" // 负场数
2794     WinRate float64 `json:"win_rate" // 胜率(百分比)
2795     MaxWinStreak int `json:"max_win_streak" // 最高连胜
2796 }
2797
2798 type VenueCheckinInfo struct {
2799     Id int64 `json:"id"
2800     VenueId int64 `json:"venue_id"

```

```

2801     VenueName string `json:"venue_name"
2802     CreatedAt string `json:"created_at"
2803 }
2804
2805 type VenueIdReq struct {
2806     VenueId int64 `json:"venue_id"
2807 }
2808
2809 type VenueInfo struct {
2810     Id      int64 `json:"id"
2811     Name    string `json:"name"
2812     Address string `json:"address"
2813     City    string `json:"city"
2814     District string `json:"district"
2815     Latitude float64 `json:"latitude"
2816     Longitude float64 `json:"longitude"
2817     Phone    string `json:"phone"
2818     Images   []string `json:"images"
2819     BusinessHours string `json:"business_hours"
2820     TableCount int `json:"table_count"
2821     PriceRange string `json:"price_range"
2822     Description string `json:"description"
2823     Distance float64 `json:"distance,omitempty"
2824     CheckinCount int `json:"checkin_count"
2825     Status    int `json:"status"
2826 }
2827
2828 type WechatAppPayParams struct {
2829     Appid string `json:"appid"
2830     Partnerid string `json:"partnerid"
2831     Prepayid string `json:"prepayid"
2832     Package string `json:"package"
2833     Noncestr string `json:"noncestr"
2834     Timestamp string `json:"timestamp"
2835     Sign string `json:"sign"
2836 }
2837
2838 // File: backend/internal/utlis/http.go
2839 package utlis
2840
2841 import (
2842     "context"
2843     "net/http"
2844     "strings"
2845 )
2846
2847 // contextKey 用于在 context 中存储 request
2848 type contextKey string
2849
2850 const RequestContextKey contextKey = "http_request"

```

```

2851
2852 // GetClientIP 从 context 中获取客户端 IP
2853 func GetClientIP(ctx context.Context) string {
2854     req, ok := ctx.Value(RequestContextKey).(*http.Request)
2855     if !ok {
2856         return ""
2857     }
2858
2859     // 优先从 X-Forwarded-For 获取
2860     xff := req.Header.Get("X-Forwarded-For")
2861     if xff != "" {
2862         // 取第一个 IP
2863         ips := strings.Split(xff, ",")
2864         if len(ips) > 0 {
2865             return strings.TrimSpace(ips[0])
2866         }
2867     }
2868
2869     // 尝试 X-Real-IP
2870     xri := req.Header.Get("X-Real-IP")
2871     if xri != "" {
2872         return xri
2873     }
2874
2875     // 从 RemoteAddr 获取
2876     remoteAddr := req.RemoteAddr
2877     if idx := strings.LastIndex(remoteAddr, ":"); idx != -1 {
2878         return remoteAddr[:idx]
2879     }
2880     return remoteAddr
2881 }
2882
2883 // GetUserAgent 从 context 中获取 User-Agent
2884 func GetUserAgent(ctx context.Context) string {
2885     req, ok := ctx.Value(RequestContextKey).(*http.Request)
2886     if !ok {
2887         return ""
2888     }
2889     return req.UserAgent()
2890 }
2891
2892 // WithRequest 将 request 添加到 context
2893 func WithRequest(ctx context.Context, req *http.Request) context.Context {
2894     return context.WithValue(ctx, RequestContextKey, req)
2895 }
2896
2897 // File: backend/internal/utls/jwt_parser.go
2898 package utls
2899
2900 import (

```

```

2901     "context"
2902     "encoding/json"
2903     "errors"
2904     "fmt"
2905 )
2906
2907 // GetUserIDFromCtx extracts the user ID from the go-zero context.
2908 // It handles different numeric types that might be present after JWT decoding.
2909 func GetUserIDFromCtx(ctx context.Context) (int64, error) {
2910     var userID int64
2911     var err error
2912
2913     val := ctx.Value("user_id") // In this project, the key is "user_id"
2914     if val == nil {
2915         return 0, errors.New("unauthorized: user_id not found in context")
2916     }
2917
2918     switch v := val.(type) {
2919     case json.Number:
2920         userID, err = v.Int64()
2921         if err != nil {
2922             return 0, fmt.Errorf("failed to parse user ID from json.Number: %w", err)
2923         }
2924     case float64:
2925         userID = int64(v)
2926     case int64:
2927         userID = v
2928     default:
2929         return 0, fmt.Errorf("invalid userID type '%T' in context", v)
2930     }
2931
2932     if userID <= 0 {
2933         return 0, errors.New("invalid user ID in context")
2934     }
2935
2936     return userID, nil
2937 }
2938
2939 // GetOptionalUserIDFromCtx extracts user_id from context when present.
2940 // It returns 0 with nil error for anonymous requests.
2941 func GetOptionalUserIDFromCtx(ctx context.Context) (int64, error) {
2942     userID, err := GetUserIDFromCtx(ctx)
2943     if err != nil {
2944         return 0, nil
2945     }
2946     return userID, nil
2947 }
2948
2949 // GetAdminIDFromCtx extracts the admin ID from the context.
2950 // Admin IDs are stored as negative values in JWT to distinguish from regular users.

```

```

2951 func GetAdminIDFromCtx(ctx context.Context) (uint64, error) {
2952     var userID int64
2953     var err error
2954
2955     val := ctx.Value("user_id")
2956     if val == nil {
2957         return 0, errors.New("unauthorized: user_id not found in context")
2958     }
2959
2960     switch v := val.(type) {
2961     case json.Number:
2962         userID, err = v.Int64()
2963         if err != nil {
2964             return 0, fmt.Errorf("failed to parse admin ID from json.Number: %w", err)
2965         }
2966     case float64:
2967         userID = int64(v)
2968     case int64:
2969         userID = v
2970     default:
2971         return 0, fmt.Errorf("invalid userId type '%T' in context", v)
2972     }
2973
2974     // Admin ID is stored as negative
2975     if userID >= 0 {
2976         return 0, errors.New("invalid admin ID in context")
2977     }
2978
2979     return uint64(-userID), nil
2980 }
2981
2982 // File: backend/internal/utis/password.go
2983 package utis
2984
2985 import (
2986     "golang.org/x/crypto/bcrypt"
2987 )
2988
2989 // HashPassword 密码加密
2990 func HashPassword(password string) (string, error) {
2991     bytes, err := bcrypt.GenerateFromPassword([]byte(password), bcrypt.DefaultCost)
2992     return string(bytes), err
2993 }
2994
2995 // CheckPasswordHash 验证密码
2996 func CheckPasswordHash(password, hash string) bool {
2997     err := bcrypt.CompareHashAndPassword([]byte(hash), []byte(password))
2998     return err == nil
2999 }
3000

```